

AWS System Design Handbook

Scalable, Resilient, and
Secure Architectures



AUTHOR

MANISH KUMAR

AWS System Design Handbook: Scalable, Resilient, and Secure Architectures

Table of Contents

	1. Introduction to AWS System Design.....	18
	What Is System Design in the Cloud?	18
	Why AWS for System Design?	18
	The Role of an AWS System Architect.....	19
	Traditional vs AWS Architecture	19
	Key AWS Services Used in System Design	20
	Example: Create a Simple VPC and EC2 using Terraform	20
	Example: Launching EC2 using AWS CLI	21
	Example: Serverless Hello World (Lambda + API Gateway).....	21
	Lambda Handler in Python.....	21
	Deploy using AWS CLI.....	22
	SDK Example: Creating an S3 Bucket in Python (Boto3)	22
	Takeaway Checklist.....	22
	2. Core Concepts of AWS Architecture	23
	AWS Global Infrastructure: The Foundation	23
	Key Terms:.....	23
	CLI Example: List AWS Regions and Availability Zones.....	23
	High Availability (HA).....	23
	Design Strategy:.....	24
	Terraform Snippet: High Availability EC2 via ASG	24
	Fault Tolerance (FT)	24
	Examples:.....	24
	SDK Example: Enable S3 Versioning.....	24
	Scalability	25
	Auto Scaling Group with CLI.....	25
	Elasticity	25
	Tools for Elastic Systems:.....	25
	Loose Coupling.....	26
	Design with Loose Coupling:	26

IaC Example: SQS Queue for Decoupling.....	26
🧱 Microservices	26
Tools to Build Microservices on AWS:.....	26
🔧 Real-World Mapping: Apply Concepts in a Real Architecture	27
✅ Checklist: Apply These Concepts in Every Design.....	27
🧠 Pro Tip from the Field	27
⚙️ 3. Foundational AWS Services for System Design	28
🚀 A. Compute Services.....	28
1. Amazon EC2 (Elastic Compute Cloud)	28
2. AWS Lambda	29
3. Amazon ECS / Fargate / EKS	30
💾 B. Storage Services	30
1. Amazon S3 (Simple Storage Service).....	30
2. Amazon EBS (Elastic Block Store)	31
3. Amazon EFS (Elastic File System)	32
🌐 C. Networking Services.....	32
1. Amazon VPC (Virtual Private Cloud)	32
2. Elastic Load Balancer (ELB)	33
3. Route 53	33
📄 D. Database Services.....	34
1. Amazon RDS (Relational Database Service)	34
2. Amazon DynamoDB.....	34
3. Amazon Aurora	35
🔍 Real-World Mapping: Video Platform Example	35
✅ Summary Table: Foundation Services	35
🧠 Pro Tip from Architect:	36
🔒 4. Security, Identity & Compliance in AWS Architecture	37
⚙️ A. AWS Shared Responsibility Model	37
👤 B. Identity and Access Management (IAM).....	37
🔒 IAM Components:	37
🧱 IAM Policy Example (Read-only S3).....	37
🔧 Terraform: IAM Role for EC2	38

🔑 C. Secrets Management	38
🔒 Use:	38
💻 CLI Example: Create Secret.....	39
🧠 SDK Example: Retrieve Secret (Python)	39
🛡️ D. Network Security	39
🔒 Security Groups vs Network ACLs	39
🔧 Terraform: Secure Web Server Group	39
🚀 E. Encryption and KMS.....	40
🔒 Encryption Options:.....	40
💻 CLI: Encrypt EBS Volume.....	40
⚙️ F. AWS Organizations & SCPs (Service Control Policies)	41
🧠 Why Use It?	41
🛡️ SCP Example: Deny EC2 Launch in Non-Approved Regions	41
🕵️ G. Logging, Monitoring, and Auditing.....	41
🔍 Tools:	41
💻 Enable CloudTrail:	42
🧠 H. Zero Trust Architecture (ZTA) on AWS	42
🔒 Design Patterns:.....	42
🧠 I. Compliance Frameworks and Tools	42
🔍 Tools:	42
🧠 Real-World Secure Architecture (Example).....	43
✅ Security Best Practices Checklist.....	43
🧠 Pro Tip from the Field	43
⚙️ 5. Designing Scalable and Resilient Architectures on AWS	44
🏠 A. Horizontal vs Vertical Scaling.....	44
🚀 B. Elastic Load Balancing (ELB)	44
🔄 Types of ELB:.....	44
🔧 Example: Create ALB with CLI	45
🔧 C. Auto Scaling Groups (ASG).....	45
🧠 Scaling Triggers:.....	45

🔧 Terraform: ASG for EC2 Web Tier	45
🚀 D. Multi-AZ & Multi-Region Architectures	46
📍 Multi-AZ (Availability Zone):.....	46
🌍 Multi-Region:	46
🍳 E. Decoupling via Queues, Topics, Streams	46
🔧 Tools:	46
🔧 Example: SQS Queue with Terraform	47
📦 F. Stateless Design & Session Management	47
Approaches:	47
✅ Use Case Example:.....	47
📁 G. Caching for Performance and Resilience	47
Caching Layers:	48
🔄 H. Retry Logic, Backoff, and Idempotency	48
🧠 Key Patterns:.....	48
Python Example: Safe Retry Logic	48
🔄 I. Failover & Disaster Recovery Strategies	49
Recovery Patterns:.....	49
📊 J. Real-World Scalable Architecture Example (E-Commerce)	49
✅ Best Practices Summary	50
📁 6. Data Storage & Database Selection in AWS	51
📦 A. AWS Storage Options – The 3 Core Models	51
♦ 1. Amazon S3 (Object Storage)	51
🔧 CLI: Upload a file	51
🧱 Terraform: Create a public S3 bucket.....	51
♦ 2. Amazon EBS (Block Storage).....	52
🔧 CLI: Create encrypted EBS volume	52
♦ 3. Amazon EFS (File Storage).....	52
🔧 CLI: Create EFS + Mount Targets	52
🚀 B. AWS Database Models	53
📦 C. Amazon RDS (Relational DB Service)	53
✅ Key Features:	53

 Terraform: Create a PostgreSQL DB	53
 D. Amazon DynamoDB (NoSQL, Key-Value/Document)	54
Use cases:	54
 CLI: Create table	54
 IAM Policy to access DynamoDB	54
 E. Global and Multi-Region Data	55
Options:	55
 S3 CRR: Replicate objects to another region	55
 F. Backup, Retention, and Archiving	55
Tools:	55
 CLI: Move S3 files to Glacier	55
 G. Analytics-Ready Storage	55
Athena Example:	56
 H. Data Access & Security Considerations	56
 I. Real-World Storage Strategy Example	56
 Best Practices Summary	57
 7. Networking & Connectivity Design in AWS	58
 A. Virtual Private Cloud (VPC) – The Networking Foundation	58
 VPC Structure Example:	58
 Terraform: Create a basic VPC	59
 CLI: Create an internet gateway	59
 B. Network Security – SGs and NACLs	59
 C. Internet Access – IGW and NAT	59
 NAT Gateway Creation (CLI)	60
 D. Internal Connectivity – VPC Peering, Transit Gateway	60
1. VPC Peering	60
 CLI: VPC Peering Request	60
2. Transit Gateway (TGW)	60
 E. Hybrid Connectivity – VPN, Direct Connect, SD-WAN	60
 CLI: Create VPN connection	61
 F. Route 53 – DNS & Routing Control	61

 CLI: Create a public DNS zone.....	61
 G. PrivateLink, VPC Endpoints, and Service Access	62
 Create Gateway Endpoint to access S3 privately	62
 H. Monitoring, Logging, and Traffic Flow	62
 Enable Flow Logs for a VPC	62
 I. Sample Architecture – Secure Public Web App	63
 Best Practices Summary	63
 8. Security, Identity & Compliance in AWS	64
 A. The Shared Responsibility Model.....	64
 B. IAM – Identity & Access Management	64
Key Concepts:	64
 CLI: Create an IAM user with programmatic access	64
 Terraform: Create IAM policy + role.....	65
 C. Encryption – At Rest and In Transit	65
 CLI: Create a KMS CMK	66
 D. Secrets Management	66
Options:	66
 CLI: Store secret in Secrets Manager.....	66
 Terraform: Create secret with rotation	66
 E. Network Security – SGs, NACLs, WAF, Shield, Firewall Manager	66
 CLI: Deploy AWS WAF Web ACL with IP block	67
 F. Identity Federation – SSO & External IdP.....	67
Terraform: AWS IAM Identity Provider	67
 G. Compliance & Governance – Guardrails.....	68
SCP Example: Deny All Except EC2 + S3	68
 H. Security Monitoring Tools	68
 I. Real-World Use Case: Secure Microservice Architecture	69
 Best Practices Summary	69
 9. Observability & Monitoring Architecture in AWS	70
 A. The AWS Observability Stack	70
 B. CloudWatch – Metrics, Alarms, Dashboards	70

 1. Metrics.....	70
 CLI: Push a custom metric	71
 2. Alarms	71
 Terraform: CPU Alarm with SNS Notification	71
 3. Dashboards	71
 C. CloudWatch Logs – Centralized Logging	72
◆ Integrate Logs From:	72
 CLI: Create log group and stream	72
 Terraform: CloudWatch Agent Configuration	72
 D. Tracing with AWS X-Ray	72
 SDK: Add X-Ray to a Python Lambda	73
 E. Container Observability (ECS/EKS)	73
ECS (Fargate or EC2):.....	73
EKS:	74
 F. Global Event Monitoring – EventBridge.....	74
Example:	74
 G. Cost-Aware Monitoring.....	74
 H. Use Case Example: Production-Grade Observability Stack	75
 Best Practices Summary	75
 10. Designing for Resilience, Availability & Fault Tolerance	76
 A. Core Principles	76
 B. Multi-AZ & Multi-Region Architecture.....	76
◆ Multi-AZ:	76
◆ Multi-Region:	76
 Example: Multi-AZ Load Balanced Web App	77
 C. Key AWS Services Supporting Resilience.....	78
 D. Self-Healing Infrastructure.....	78
1. EC2 Auto Recovery with CloudWatch Alarm.....	78
2. ASG with Instance Health Check.....	78
 E. Backup & Recovery Strategy	79
Terraform: Backup Vault + Backup Plan	79

	F. Designing for Graceful Degradation	79
	G. Disaster Recovery Strategies.....	80
	Best Practices Summary	80
	Bonus Tools for Resilience Testing	81
	11. Designing Cost-Optimized Architectures in AWS.....	82
	A. Core Principles of Cost Optimization	82
	B. EC2 & Compute Optimization	82
◆	Instance Selection.....	82
◆	Right-Sizing with Compute Optimizer	82
◆	Spot Instances for Flexible Workloads	83
	C. Storage Optimization	83
◆	S3 Storage Classes.....	83
◆	EBS Volume Rightsizing	84
	D. Serverless Cost Control	84
◆	Lambda	84
◆	Example: Lambda Cost Estimation	84
	E. Database Cost Optimization.....	85
◆	RDS.....	85
◆	DynamoDB	85
	F. Networking Optimization.....	85
◆	Avoid NAT Gateway Overuse.....	85
	G. Monitoring & Budgeting Tools	86
	Create Budget.....	86
	Sample JSON:	86
	H. Containers Cost Optimization (ECS, EKS).....	86
	I. Scheduled Scaling & Shutdowns	87
	Scheduled Auto Scaling Policy	87
	Best Practices Summary	88
	12. Security, Compliance & Identity in AWS Architecture.....	89
	A. AWS Shared Responsibility Model	89
	B. Identity & Access Management (IAM)	89

◆ Core Best Practices	89
🔧 IAM Role for EC2 to Access S3 (Terraform)	89
🔒 C. Data Protection & Encryption	90
◆ At Rest	90
◆ In Transit	91
🏠 D. Network Security	91
◆ Security Groups vs. NACLs.....	91
◆ Best Practices.....	91
🏠 E. WAF, Shield & GuardDuty.....	92
◆ WAF Rule for Blocking SQL Injection.....	92
📋 F. Compliance & Governance	92
◆ Tools for Compliance	92
🔧 G. Monitoring, Logging & Alerts	93
◆ CloudTrail.....	93
◆ CloudWatch Logs + Alarms	93
✅ Best Practices Summary	94
🔧 13. Monitoring, Logging & Observability in AWS Architecture	95
🧠 A. What is Observability?	95
🇮🇹 B. CloudWatch: Metrics, Dashboards, and Alarms	95
◆ Metrics Collection	95
🔧 Custom Metrics (CLI)	96
💻 Dashboards (Terraform)	96
🔔 Alarms Example	96
📁 C. Logging with CloudWatch Logs.....	97
◆ Log Sources	97
🔧 Setup (for EC2 Linux).....	97
⚙️ D. Distributed Tracing with AWS X-Ray.....	97
🔧 Example: Tracing in Lambda (Node.js)	98
🔔 E. Event-Driven Observability: EventBridge	98
◆ Example: Triggering Lambda on EC2 Stop.....	98

📁	F. Centralized Logging with CloudTrail & S3	99
◆	CloudTrail – Capture API Calls Across AWS	99
🗺️	G. Third-Party Integration	99
✅	Best Practices Summary	99
🔧	Example: Observability Stack for a Real-World Web App	100
🔵	15. Disaster Recovery (DR) & High Availability (HA) Strategies	101
📋	A. Key Concepts	101
🧠	B. Availability Zones & Regions Strategy	101
◆	Availability Zones (AZs).....	101
◆	Multi-Region Strategy	102
🔄	C. DR Patterns & Use Cases.....	102
🔧	Example: Pilot Light for RDS + EC2	102
📦	D. AWS Services for DR & HA	103
🔧	E. Elastic Disaster Recovery (AWS DRS)	103
📄	F. Backup Strategies	104
🔍	G. Monitoring DR Health	104
✅	Best Practices Summary	104
⚙️	16. CI/CD & DevOps Best Practices in AWS System Design	106
🧠	A. What Is CI/CD?	106
🚀	B. DevOps Tools in AWS	106
🔧	C. CI/CD Architecture in AWS	107
🧱	Example Components:	107
🔧	Example: Terraform Pipeline (CodeCommit to Lambda)	107
🧪	D. Testing Strategies	109
🔄	E. Deployment Strategies	109
	1. Blue/Green Deployment.....	109
	2. Canary Deployment	109
	3. Rolling Deployment.....	110
🔒	F. Security in CI/CD	110
🎯	G. DevOps Monitoring & Rollback	110
✅	Best Practices Summary	111

	17. Performance Optimization in AWS System Design.....	112
	A. Why Performance Optimization Matters	112
	B. Key Optimization Areas	112
	C. Compute Optimization	112
◆	EC2 Right-Sizing	112
◆	Lambda Performance Tuning.....	113
	D. Storage Optimization	114
	E. Database Optimization	114
◆	Amazon RDS	114
◆	Amazon DynamoDB.....	115
	F. Network Optimization	115
	G. Caching Optimization	115
	H. Observability for Optimization	116
	Best Practices Summary	117
	18. Observability & Logging Architecture in AWS	118
	A. Three Pillars of Observability	118
	B. Metrics Collection	118
◆	Native Metrics with CloudWatch.....	118
	Infrastructure-as-Code (Terraform):	119
	C. Logging Strategies.....	119
◆	CloudWatch Logs (Default)	119
◆	S3-Based Logging (Long-Term).....	120
◆	OpenSearch (Log Analytics)	120
	D. Tracing & Distributed Monitoring.....	120
◆	AWS X-Ray	120
◆	OpenTelemetry	121
	E. Dashboards and Visualization.....	121
◆	CloudWatch Dashboards.....	121
◆	Third-Party Tools Integration	122
	F. Alerting and Incident Response.....	122

◆ CloudWatch Alarms + SNS	122
◆ Automation with Lambda or SSM	122
📊 G. Observability Architecture Example	123
✅ Best Practices Summary	123
🌐 19. Designing for Global Availability and Multi-Region Deployments	124
🧠 Why Multi-Region Architecture?.....	124
🏠 A. Core AWS Services Used for Global Design	124
🌐 B. DNS Routing & Global Traffic Control	125
1. Amazon Route 53	125
2. Latency-Based Routing	125
🚀 C. Content Delivery & Network Acceleration	126
1. Amazon CloudFront.....	126
2. AWS Global Accelerator	126
📡 D. Data Replication Strategies.....	126
1. S3 Cross-Region Replication (CRR).....	126
2. Aurora Global Database	127
3. DynamoDB Global Tables	127
💡 E. Disaster Recovery (DR) Strategies.....	127
💡 Use AWS Elastic Disaster Recovery (DRS).....	127
⚙️ F. Infrastructure Deployment & Sync	128
1. CloudFormation StackSets	128
2. CI/CD Deployment Strategy	128
🔄 G. Synchronizing State & Handling Failover	128
🧠 H. Common Design Patterns	129
🌐 Active-Active Web App with Global Load Balancing.....	129
🏠 Global Microservices with Regional Failover	129
✅ Best Practices Summary	129
🌱 20. Real-World System Design Patterns & Architectures in AWS	130
🔄 A. Event-Driven Architecture (EDA).....	130
◆ When to Use	130
◆ AWS Services Involved	130

💡 Architecture Diagram	130
🔧 Terraform Snippet	131
🧩 B. CQRS + Event Sourcing	131
♦ When to Use	131
💡 Design Pattern	131
🔧 AWS Implementation	131
🧠 C. Serverless Microservices	132
♦ When to Use	132
🔧 Common Stack	132
📝 Example CLI Setup	132
📦 D. E-Commerce Web Architecture	132
♦ Requirements	132
🏠 AWS Blueprint	133
🔧 Terraform Snippet for S3 Static Website + CloudFront	133
📺 E. Real-Time Streaming with Analytics	133
♦ Use Cases	134
🏠 AWS Architecture	134
🔧 AWS CLI Kinesis Example	134
🔒 F. Zero Trust Security Architecture	134
♦ AWS Implementation	134
⚙️ G. Hybrid Cloud / On-Prem + AWS	134
♦ Use Cases	135
🏠 AWS Tools	135
🧩 H. Service Mesh with AWS App Mesh	135
♦ When to Use	135
🔧 Key Features	135
👤 I. Multi-Tenant SaaS Platform	135
♦ Use Cases	135
🏠 AWS Design	136
✅ Summary: Choose the Right Pattern Based on Your Needs	136

 21. Case Studies & Real-World Architecture Breakdowns	137
 A. E-Commerce Platform: Scaling for Black Friday	137
 Client: Mid-size Retailer	137
 Goal: Handle massive seasonal traffic spikes (20x) during flash sales	137
 Requirements:	137
 Architecture:	137
 Flow Diagram:	138
 Challenge Solved:	138
 B. Video Streaming Startup: Global Media Delivery	138
 Client: SaaS OTT Platform	138
 Goal: Deliver low-latency video globally with analytics insights	138
 Requirements:	138
 Architecture:	139
 Streaming Flow:	139
 Challenge Solved:	139
 C. FinTech App: Secure, Compliant Multi-Tenant SaaS	139
 Client: B2B SaaS FinTech Provider	139
 Goal: Offer financial dashboards to banks with complete tenant isolation and regulatory compliance	139
 Requirements:	139
 Architecture:	140
 Security Practices:	140
 Challenge Solved:	140
 D. Healthcare Analytics: HIPAA-Compliant Data Lake	140
 Client: National Hospital Chain	140
 Goal: Collect, store, and analyze sensitive patient data securely across 80+ clinics	141
 Requirements:	141
 Architecture:	141
 Security Controls:	141
 Challenge Solved:	141
 E. Multi-Region SaaS Platform: Always-On Uptime	141

 Client: Global SaaS CRM Provider	142
 Goal: Ensure zero downtime with multi-region failover & fast user experience	142
 Requirements:	142
 Architecture:	142
 Global Setup Flow:	142
 Challenge Solved:	142
 Lessons Learned Across Case Studies	143
 22. AWS System Design Architect – One-Liner Interview Q&A	144
 Glossary	152



1. Introduction to AWS System Design

System design on AWS goes beyond simply launching EC2 instances and storing data in S3. It involves **thoughtful engineering of resilient, secure, scalable, and cost-efficient architectures**, tailored for the cloud-native world. This tutorial aims to equip you with a deep understanding of designing such systems using AWS tools and best practices.



What Is System Design in the Cloud?

System design refers to the **architectural blueprint of how components of an application interact** — including compute, networking, storage, security, and availability.

In the cloud, this expands to include:

- **Dynamic scaling**
- **Managed services (e.g., RDS, Lambda)**
- **Global availability**
- **Infrastructure as Code**
- **Cost awareness**



Why AWS for System Design?

Amazon Web Services (AWS) is the world's leading cloud provider, offering 200+ fully managed services. It enables:

- **Rapid provisioning** of infrastructure
- **Global distribution** of applications
- **Advanced security** controls
- **Highly available designs**
- **Economies of scale**

✅ **Fun Fact:** AWS has more services and more features within those services than any other cloud provider—ranging from infrastructure technologies like compute, storage, and databases—to emerging technologies such as AI/ML and IoT.



The Role of an AWS System Architect

An AWS System Architect is responsible for:

- Understanding business requirements
- Selecting the right AWS services
- Designing high-level and low-level architecture
- Implementing automation (e.g., Terraform, CloudFormation)
- Applying Well-Architected Framework principles
- Cost governance and observability

vs Traditional vs AWS Architecture

Feature	Traditional Infra	AWS System Design
Provisioning	Manual	Automated via CLI/IaC
Capacity Planning	Static	Auto Scaling & Serverless
High Availability	Complex & Expensive	Built-in with Multi-AZ/Region Support
Cost Structure	CapEx	Pay-as-you-go (OpEx)
Security	Perimeter-based	Zero-trust with IAM, VPC, SCP
Monitoring	Manual Tools	Built-in: CloudWatch, X-Ray

Key AWS Services Used in System Design

Here's a foundational list:

Category	Key Services
Compute	EC2, Lambda, ECS, EKS, Fargate
Storage	S3, EBS, EFS, Glacier
Networking	VPC, ALB/NLB, Route 53, Direct Connect
Database	RDS, Aurora, DynamoDB, Redshift
Security	IAM, KMS, Secrets Manager, Cognito
DevOps/IaC	CloudFormation, Terraform, CodePipeline
Monitoring	CloudWatch, X-Ray, CloudTrail

Example: Create a Simple VPC and EC2 using Terraform

```
# main.tf

provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "main" {
  vpc_id      = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
}

resource "aws_instance" "web" {
```

```

ami            = "ami-0c55b159cbfafelf0" # Amazon Linux 2 AMI
instance_type  = "t2.micro"
subnet_id      = aws_subnet.main.id

tags = {
    Name = "WebServer"
}

```

Run with:

```

terraform init
terraform apply

```



Example: Launching EC2 using AWS CLI

```

aws ec2 run-instances \
  --image-id ami-0c55b159cbfafelf0 \
  --count 1 \
  --instance-type t2.micro \
  --subnet-id subnet-12345678 \
  --tag-specifications
'ResourceType=instance,Tags=[{Key=Name,Value=WebServer}]'

```



Example: Serverless Hello World (Lambda + API Gateway)

Lambda Handler in Python

```

def lambda_handler(event, context):
    return {
        'statusCode': 200,
        'body': 'Hello from Lambda!'
    }

```

Deploy using AWS CLI

```
zip function.zip lambda_function.py

aws lambda create-function \
  --function-name HelloWorld \
  --runtime python3.11 \
  --role arn:aws:iam::123456789012:role/lambda-ex \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://function.zip
```

SDK Example: Creating an S3 Bucket in Python (Boto3)

```
import boto3

s3 = boto3.client('s3')
bucket_name = "my-system-design-demo-bucket"

s3.create_bucket(Bucket=bucket_name)
print(f"Bucket '{bucket_name}' created.")
```

Takeaway Checklist

Item	Description
Define your core requirements	Latency, scaling, durability, security
Understand AWS service offerings	Match services with requirements
Automate Infrastructure	Use Terraform or CloudFormation
Design for failure	Multi-AZ, retries, health checks
Implement security best practices	IAM roles, KMS, least privilege
Monitor and adapt	CloudWatch, X-Ray, custom dashboards



2. Core Concepts of AWS Architecture

Designing effective systems in AWS means mastering a set of cloud-native architecture principles that power resilience, efficiency, and scalability. These concepts are the **building blocks** that govern how you should structure services, connect resources, and design for failure and growth.



AWS Global Infrastructure: The Foundation

Before applying architecture principles, you need to understand the **global layout of AWS**.



Key Terms:

Component	Description
Region	A physical location with multiple data centers (AZs)
Availability Zone (AZ)	Isolated data centers within a region
Edge Location	CDN endpoints for caching via CloudFront
Local Zones / Wavelength Zones	For ultra-low-latency computing



CLI Example: List AWS Regions and Availability Zones

```
aws ec2 describe-regions --output table
aws ec2 describe-availability-zones --region us-east-1 --output table
```



Design Tip: Always use Multi-AZ architectures for critical services. Use Multi-Region for global resilience or compliance.



High Availability (HA)

High availability means your system stays **accessible** even when parts of it fail.

Design Strategy:

- Use **Elastic Load Balancers** across multiple AZs
- Replicate data across **Multi-AZ databases**
- Architect with **auto healing** components

Terraform Snippet: High Availability EC2 via ASG

```
resource "aws_autoscaling_group" "web_asg" {
  availability_zones = ["us-east-1a", "us-east-1b"]
  desired_capacity   = 2
  max_size           = 4
  min_size           = 1
  launch_configuration = aws_launch_configuration.web_lc.name
}
```

Fault Tolerance (FT)

Fault tolerance ensures that **failures are isolated** and the system **continues to function**.

Examples:

- Use **Elastic Load Balancers** to detect failed instances
- Set **retry logic and circuit breakers** in app code
- Store redundant data across **multiple services or regions**

SDK Example: Enable S3 Versioning

```
import boto3

s3 = boto3.client('s3')
s3.put_bucket_versioning(
    Bucket='my-critical-bucket',
    VersioningConfiguration={'Status': 'Enabled'}
)
```

🔗 Design Tip: Use **retry** + **exponential backoff** in SDK/API calls (especially for transient failures).



Scalability

Scalability is the system's ability to **handle increasing workloads** by:

- **Vertical scaling:** upgrading instance types (e.g., t2.micro → m5.large)
- **Horizontal scaling:** adding more instances or Lambda concurrency

Auto Scaling Group with CLI

```
aws autoscaling create-auto-scaling-group \  
  --auto-scaling-group-name my-asg \  
  --launch-configuration-name my-launch-config \  
  --min-size 2 --max-size 6 --desired-capacity 3 \  
  --vpc-zone-identifier "subnet-abc123,subnet-def456"
```

✅ AWS Tip: Use **DynamoDB On-Demand** and **Lambda** to auto-scale without infrastructure management.



Elasticity

Elasticity is about **automatically provisioning and deprovisioning resources** based on demand.

Tools for Elastic Systems:

- **Auto Scaling + Target Tracking** for EC2
- **Lambda** (serverless auto-scaling)
- **Aurora Serverless, DynamoDB** for elastic databases
- **SQS + Lambda** for decoupled auto-scaling compute

Loose Coupling

Tightly coupled systems break under pressure. Loose coupling increases resilience and allows parts to evolve independently.

Design with Loose Coupling:

- Use **SQS, SNS, EventBridge** for asynchronous communication
- Deploy **microservices** independently
- Store **shared state in S3, DynamoDB**, not in app memory

IaC Example: SQS Queue for Decoupling

```
resource "aws_sqs_queue" "task_queue" {  
  name = "task-processing-queue"  
}
```

Microservices

Microservices are a **natural fit for AWS**, with each service responsible for a single function and communicating over APIs.

Tools to Build Microservices on AWS:

- **API Gateway + Lambda**
- **ECS or EKS** for containerized services
- **App Mesh or Service Discovery** for traffic management

 Bonus: Microservices can scale independently and enforce **least privilege IAM policies per service**.



Real-World Mapping: Apply Concepts in a Real Architecture

Let's say we want to design a **Video Processing Platform**. Here's how core concepts apply:

Component	AWS Service Used	Concept Applied
Upload Endpoint	API Gateway + Lambda	Elasticity, Serverless
File Storage	S3 + Event Notification	Durability, Loose Coupling
Video Processing	Lambda + Step Functions	Microservices, Fault Tolerance
Metadata Store	DynamoDB	Scalability, Elasticity
Distribution	CloudFront	Edge Caching, HA
Monitoring	CloudWatch, X-Ray	Observability



Checklist: Apply These Concepts in Every Design

 Principle	How to Apply in AWS
High Availability	Use Multi-AZ load balancers, RDS, etc.
Fault Tolerance	Use retries, failovers, S3 versioning
Scalability	Use ASGs, Lambda, DynamoDB
Elasticity	Use serverless or scaling policies
Loose Coupling	Use SQS, SNS, EventBridge
Microservices	Use containerized services, Lambda/APIGW



Pro Tip from the Field

"Architecting for failure is **not optional** in the cloud. If you're not expecting components to fail, you're designing for disaster."

3. Foundational AWS Services for System Design

AWS provides a comprehensive suite of services that form the building blocks of every cloud-native architecture. In this section, we'll **describe each foundational category in depth**—Compute, Storage, Networking, and Databases—with **CLI, SDK, and Terraform examples** to show how these services are used in real-world system design.

A. Compute Services

1. Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 offers **resizable compute capacity** in the cloud. It's the go-to choice for workloads requiring OS-level control.

Key Features

- Instance types (general, compute, memory optimized)
- Auto Scaling Groups
- Elastic Load Balancing
- Spot, On-Demand, Reserved pricing

Terraform Example

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "WebServer"
  }
}
```

CLI Example

```
aws ec2 run-instances \  
  --image-id ami-0c55b159cbfafa1f0 \  
  --count 1 \  
  --instance-type t2.micro \  
  --tag-specifications  
'ResourceType=instance,Tags=[{Key=Name,Value=WebServer}]'
```

2. AWS Lambda

A **serverless compute** service that runs your code without provisioning or managing servers.

Key Use Cases

- REST APIs
- Event-driven workloads
- Cron jobs
- Lightweight microservices

Code Example: Python Lambda

```
def lambda_handler(event, context):  
    return {  
        'statusCode': 200,  
        'body': 'Hello from Lambda!'  
    }
```

CLI Deployment

```
zip function.zip lambda_function.py  
  
aws lambda create-function \  
  --function-name HelloWorld \  
  --runtime python3.11 \  
  --
```

```
--role arn:aws:iam::123456789012:role/lambda-ex \  
--handler lambda_function.lambda_handler \  
--zip-file fileb://function.zip
```

3. Amazon ECS / Fargate / EKS

- **ECS (Elastic Container Service):** AWS-native container orchestration.
- **Fargate:** Serverless containers (no EC2 required).
- **EKS (Elastic Kubernetes Service):** Managed Kubernetes.

IaC: ECS Cluster with Fargate Task

```
resource "aws_ecs_cluster" "demo" {  
  name = "demo-cluster"  
}  
  
resource "aws_ecs_task_definition" "nginx" {  
  family           = "nginx-task"  
  requires_compatibilities = ["FARGATE"]  
  network_mode     = "awsvpc"  
  cpu              = "256"  
  memory           = "512"  
  container_definitions = jsonencode([  
    {  
      name       = "nginx"  
      image      = "nginx"  
      essential  = true  
      portMappings = [  
        {  
          containerPort = 80  
        }  
      ]  
    }  
  ])  
}
```

B. Storage Services

1. Amazon S3 (Simple Storage Service)

Highly scalable object storage for data backup, content delivery, analytics, etc.

Features

- Lifecycle rules
- Versioning
- Server-side encryption (SSE-S3, SSE-KMS)
- Event notifications

CLI Example

```
aws s3 mb s3://my-app-data
aws s3 cp file.txt s3://my-app-data/
```

Terraform Example

```
resource "aws_s3_bucket" "app_data" {
  bucket = "my-app-data"

  versioning {
    enabled = true
  }

  lifecycle_rule {
    enabled = true
    expiration {
      days = 30
    }
  }
}
```

2. Amazon EBS (Elastic Block Store)

Block storage for EC2 instances.

CLI Example

```
aws ec2 create-volume \  
  --availability-zone us-east-1a \  
  --size 20 \  
  --volume-type gp3
```

3. Amazon EFS (Elastic File System)

Fully managed **shared file system**, supports concurrent access from multiple EC2 instances.

Mount from EC2:

```
sudo mount -t nfs4 -o nfsvers=4.1 fs-xxxx.efs.us-east-1.amazonaws.com:/  
/mnt/efs
```

C. Networking Services

1. Amazon VPC (Virtual Private Cloud)

Your **isolated network in AWS**. Core to secure architecture.

Components:

- Subnets (public/private)
- Internet Gateway (IGW)
- NAT Gateway
- Security Groups (SG) & Network ACLs (NACLs)
- Route Tables

Terraform Snippet

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "public" {
```

```
vpc_id            = aws_vpc.main.id
cidr_block        = "10.0.1.0/24"
map_public_ip_on_launch = true
}
```

2. Elastic Load Balancer (ELB)

Distributes traffic across targets:

- **ALB:** For HTTP/HTTPS
- **NLB:** For TCP
- **GLB:** For global traffic (coming soon)

CLI Example

```
aws elbv2 create-load-balancer \
  --name my-alb \
  --subnets subnet-12345678 subnet-23456789 \
  --scheme internet-facing \
  --type application
```

3. Route 53

Scalable DNS and domain name registration service.

Use Cases:

- Custom domains
- Failover routing
- Geo-routing

CLI Example: Create Hosted Zone

```
aws route53 create-hosted-zone \
  --name mydomain.com \
  --caller-reference "unique-ref-001"
```



D. Database Services

1. Amazon RDS (Relational Database Service)

Managed MySQL, PostgreSQL, MariaDB, SQL Server, Oracle.



Terraform Example

```
resource "aws_db_instance" "mydb" {  
  engine           = "mysql"  
  instance_class   = "db.t3.micro"  
  name             = "appdb"  
  username         = "admin"  
  password         = "password123"  
  allocated_storage = 20  
  skip_final_snapshot = true  
}
```

2. Amazon DynamoDB

Fully managed, NoSQL database for fast and predictable performance.



Key Features

- Auto-scaling
- Global tables
- Streams
- TTL (Time To Live)



CLI Example

```
aws dynamodb create-table \  
  --table-name Users \  
  --attribute-definitions AttributeName=UserId,AttributeType=S \  
  --key-schema AttributeName=UserId,KeyType=HASH \  
  --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5
```

```
--billing-mode PAY_PER_REQUEST
```

3. Amazon Aurora

High-performance managed database compatible with MySQL and PostgreSQL. Supports **Aurora Serverless v2** for auto-scaling.

Why Use It?

- Up to 5x faster than MySQL
- Auto-scaling
- Cross-region replicas

Real-World Mapping: Video Platform Example

Requirement	AWS Service	Reason
Host frontend	S3 + CloudFront	Serverless static hosting
Backend APIs	Lambda + API Gateway	Scale automatically
Video uploads	S3 + Event Notifications	Event-driven processing
Processing workflow	Lambda + Step Functions	Microservice orchestration
Database	DynamoDB + S3 metadata	Scalable NoSQL and object store
Auth	Cognito	Easy user pool management
DNS	Route 53	Domain management + failover
Monitoring	CloudWatch + X-Ray	Observability

Summary Table: Foundation Services

Layer	Core AWS Services
Compute	EC2, Lambda, ECS, EKS, Fargate
Storage	S3, EBS, EFS, Glacier

Networking	VPC, Subnets, Route Tables, SGs, ELB, NAT GW
Databases	RDS, Aurora, DynamoDB, Redshift
Edge Services	CloudFront, Route 53
Monitoring	CloudWatch, X-Ray, CloudTrail
IaC/DevOps	Terraform, CloudFormation, CodePipeline

Pro Tip from Architect:

“Designing with AWS is like composing music—you must know your instruments (services), structure your notes (architecture), and balance performance with harmony (cost & security).”



4. Security, Identity & Compliance in AWS Architecture

Security and compliance in AWS isn't a feature — it's a *shared responsibility model* and an architectural discipline. As an AWS System Design Architect, your job is to implement guardrails, not gates — enabling innovation **without compromising safety**.



A. AWS Shared Responsibility Model

Responsibility	AWS	You (the customer)
Of the cloud	Hardware, networking, hypervisor	❌ You don't manage this
In the cloud	OS, application, data, IAM	✅ You fully manage this



Think of it like renting a hotel room — AWS provides the building, power, and locks; you lock the door and secure your valuables.



B. Identity and Access Management (IAM)

IAM is the **backbone of security** in AWS — controlling who can do what, and where.



IAM Components:

- **Users** – Real people (developers, admins)
- **Groups** – Collections of users (e.g., `DevOpsTeam`)
- **Roles** – For AWS services or federated access
- **Policies** – JSON-based permissions documents



IAM Policy Example (Read-only S3)

```
{
```

```
"Version": "2012-10-17",
"Statement": [{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::mybucket/*"
}]
}
```

Terraform: IAM Role for EC2

```
resource "aws_iam_role" "ec2_role" {
  name = "ec2-readonly-s3"

  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect = "Allow",
      Principal = {
        Service = "ec2.amazonaws.com"
      },
      Action = "sts:AssumeRole"
    }]
  })
}
```

✅ **Best Practice:** Use IAM Roles for services, never hardcode credentials in code or EC2 instances.

C. Secrets Management

Never store passwords, tokens, or API keys in plaintext or Git repositories.

 **Use:**

- **AWS Secrets Manager** (dynamic secrets, rotation)
- **AWS Systems Manager Parameter Store** (key/value config)

CLI Example: Create Secret

```
aws secretsmanager create-secret \  
  --name db-prod-password \  
  --secret-string '{"username":"admin","password":"Pa$$w0rd"}'
```

SDK Example: Retrieve Secret (Python)

```
import boto3  
import json  
  
client = boto3.client('secretsmanager')  
response = client.get_secret_value(SecretId='db-prod-password')  
creds = json.loads(response['SecretString'])  
print(creds['username'], creds['password'])
```

D. Network Security

Security Groups vs Network ACLs

Feature	Security Groups	Network ACLs
Stateful?	✅ Yes	❌ No (must allow return)
Applied To?	EC2, ELB, Lambda, etc.	Subnets
Direction	Inbound & Outbound	Inbound & Outbound

Terraform: Secure Web Server Group

```
resource "aws_security_group" "web_sg" {  
  name      = "web-sg"  
  vpc_id    = aws_vpc.main.id  
  
  ingress {  
    from_port = 80  
  }  
}
```

```

    to_port      = 80
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}

```

E. Encryption and KMS

AWS provides **encryption at rest and in transit** for nearly all services.

Encryption Options:

- **SSE-S3 / SSE-KMS** for S3
- **EBS encryption** via KMS
- **Aurora / RDS** supports encrypted storage
- **KMS (Key Management Service)** to manage keys

CLI: Encrypt EBS Volume

```

aws ec2 create-volume \
  --size 100 \
  --availability-zone us-east-1a \
  --volume-type gp3 \
  --encrypted \
  --kms-key-id alias/my-key

```

F. AWS Organizations & SCPs (Service Control Policies)

Organizations allow **multi-account strategy** with central management.

Why Use It?

- Isolate workloads (e.g., dev, prod, sandbox)
- Apply **central guardrails** using SCPs
- Manage billing centrally

SCP Example: Deny EC2 Launch in Non-Approved Regions

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "ec2:RunInstances",
    "Resource": "*",
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-west-2"]
      }
    }
  }]
}
```

G. Logging, Monitoring, and Auditing

Logging is key to compliance, security, and incident response.

Tools:

- **CloudTrail** – logs every API call

Written by: MANISH KUMAR

- **CloudWatch Logs** – app/service logs
- **VPC Flow Logs** – network traffic
- **GuardDuty** – threat detection
- **AWS Config** – track resource compliance and drift

Enable CloudTrail:

```
aws cloudtrail create-trail \
  --name org-trail \
  --s3-bucket-name audit-log-bucket \
  --is-multi-region-trail
```

✅ Always enable CloudTrail in **all regions**, even if unused.

H. Zero Trust Architecture (ZTA) on AWS

ZTA assumes **no network trust** — each request is verified, authenticated, and authorized.

Design Patterns:

- Use **ALB with Cognito/Auth0** for authentication
- Enable **Mutual TLS** between services
- Apply **fine-grained IAM + ABAC (Attribute-based Access Control)**
- Isolate sensitive workloads using **PrivateLink / VPC Endpoints**

I. Compliance Frameworks and Tools

AWS supports certifications such as **SOC 1/2/3, ISO 27001, HIPAA, GDPR, PCI DSS**, etc.

Tools:

- **AWS Artifact** – download compliance reports
- **AWS Audit Manager** – automate evidence collection

- **Security Hub** – centralize findings
- **Trusted Advisor** – high-level best practice checks

Real-World Secure Architecture (Example)

Layer	AWS Service	Reason
AuthN/AuthZ	Cognito + IAM + STS	User + app-level security
Network isolation	VPC, NACLs, SGs, PrivateLink	Defense-in-depth
Data encryption	KMS, SSE-S3, EBS encryption	At-rest & in-transit security
Key management	KMS, Secrets Manager	Fine-grained control
Audit trail	CloudTrail + Config + X-Ray	Incident response + compliance
Compliance automation	Audit Manager + Artifact	Continuous assurance

Security Best Practices Checklist

 Area	Best Practice
Identity	Least privilege IAM, use roles, MFA
Secrets	Use Secrets Manager, rotate frequently
Network	Use SG + NACL, private subnets, no 0.0.0.0
Encryption	Enable default encryption everywhere
Logging & Auditing	Enable CloudTrail, VPC Flow Logs, Config
Compliance	Use Artifact, Audit Manager, Security Hub
Multi-account strategy	Use AWS Organizations + SCPs
Access Control	Use ABAC, IAM Conditions, ZTA

Pro Tip from the Field

"Security must be **invisible but ever-present** — like air. Good architecture should **enable security**, not fight it."

5. Designing Scalable and Resilient Architectures on AWS

Designing a modern system on AWS requires preparing for **change, growth, and failure**. Your system must scale under pressure and gracefully recover when things go wrong — all while optimizing cost and performance.

As a System Design Architect, your goal is to design for:

- **Scalability** – Handle more users or data without rewriting the architecture.
- **Resilience** – Stay available even when parts of the system fail.
- **Fault Tolerance** – Fail gracefully and recover automatically.

A. Horizontal vs Vertical Scaling

Scaling Type	Description	AWS Services
Vertical	Increase instance size (more CPU, RAM)	RDS, EC2
Horizontal	Add more instances to distribute load	EC2 ASG, ECS, EKS

 Prefer **horizontal scaling** where possible — it's more fault-tolerant and cloud-native.

B. Elastic Load Balancing (ELB)

AWS ELB distributes incoming traffic across targets like EC2 instances, containers, or Lambda functions.

Types of ELB:

- **ALB** – Application Load Balancer (HTTP/HTTPS, Layer 7)
- **NLB** – Network Load Balancer (TCP/UDP, Layer 4)

- **CLB** – Classic Load Balancer (legacy)

Example: Create ALB with CLI

```
aws elbv2 create-load-balancer \  
  --name my-alb \  
  --subnets subnet-abc subnet-def \  
  --security-groups sg-123456
```

C. Auto Scaling Groups (ASG)

Auto Scaling ensures that your system **grows and shrinks** based on demand.

Scaling Triggers:

- CPU Utilization
- Request Count (ALB)
- Custom CloudWatch Metrics (e.g., queue depth)

Terraform: ASG for EC2 Web Tier

```
resource "aws_autoscaling_group" "web_asg" {  
  name                  = "web-asg"  
  launch_configuration  = aws_launch_configuration.web_lc.name  
  min_size              = 2  
  max_size              = 10  
  desired_capacity      = 3  
  vpc_zone_identifier   = var.public_subnets  
  
  tag {  
    key      = "Name"  
    value    = "web-instance"  
    propagate_at_launch = true  
  }  
}
```



D. Multi-AZ & Multi-Region Architectures



Multi-AZ (Availability Zone):

Designing across AZs within a region for high availability.

- RDS Multi-AZ
- ALB spanning AZs
- S3 (replicates automatically across AZs)



Multi-Region:

For global applications and disaster recovery.

- Use **Route 53 latency routing**
- Cross-region replication (S3, DynamoDB)
- Global DynamoDB tables



Always spread critical workloads across **multiple AZs**, and consider **multi-region** for disaster recovery.



E. Decoupling via Queues, Topics, Streams

To build resilient systems, decouple components so they can **fail independently**.



Tools:

- **SQS** – Message queue (decoupling)
- **SNS** – Pub/sub notifications (fan-out)
- **EventBridge** – Event bus (event-driven)
- **Kinesis** – Stream processing (real-time)

Example: SQS Queue with Terraform

```
resource "aws_sqs_queue" "task_queue" {
  name                  = "task-queue"
  visibility_timeout_seconds = 60
  message_retention_seconds = 86400
}
```

 Always use **dead letter queues (DLQ)** for fault isolation.

F. Stateless Design & Session Management

Make your services **stateless** so any instance can handle any request.

Approaches:

- Store session data in **DynamoDB**, **ElastiCache (Redis)**, or **S3**
- Use **sticky sessions** only when unavoidable

Use Case Example:

```
# Store session in DynamoDB
aws dynamodb create-table \
  --table-name user_sessions \
  --attribute-definitions AttributeName=session_id,AttributeType=S \
  --key-schema AttributeName=session_id,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

G. Caching for Performance and Resilience

Caching helps absorb traffic spikes and reduce backend load.

Caching Layers:

Layer	AWS Tool	Purpose
Edge	CloudFront	Global CDN
App	ElastiCache (Redis/Mem)	Session/data caching
DB	DAX for DynamoDB	High-performance read caching

✅ Use **CloudFront** with origin failover to S3 or another backend for max resilience.



H. Retry Logic, Backoff, and Idempotency

Even resilient systems face temporary errors. Design **clients and APIs to retry** safely.



Key Patterns:

- **Exponential Backoff** – Retry with increasing delays
- **Idempotency Tokens** – Prevent double-processing (e.g., Stripe payments)
- **Dead Letter Queues** – Capture failed events

Python Example: Safe Retry Logic

```
import boto3
import botocore
import time

client = boto3.client('dynamodb')
retries = 0

while retries < 5:
    try:
        response = client.get_item(TableName='products', Key={'id': {'S':
'123'}})
        break
    except botocore.exceptions.ClientError:
        retries += 1
```

```
time.sleep(2 ** retries) # Exponential backoff
```

I. Failover & Disaster Recovery Strategies

Recovery Patterns:

Strategy	RTO/RPO	Cost	Example
Backup & Restore	Hours	\$	S3 backups, RDS snapshots
Pilot Light	Minutes	\$\$	Minimal infra always running
Warm Standby	Seconds	\$\$\$	Scaled-down copy always on
Multi-Site Active	Near-zero	\$\$\$\$	Fully active in multiple regions

✓ Use **Route 53 + Health Checks** for DNS-based failover.

J. Real-World Scalable Architecture Example (E-Commerce)

Layer	Services Used
Frontend	CloudFront → ALB → ECS Fargate (React App)
API	ALB → Lambda (REST) or API Gateway
DB	DynamoDB (products, carts), RDS (orders)
Queues	SQS + Lambda workers for order processing
Cache	ElastiCache (Redis) for sessions & products
Auth	Cognito + IAM
Storage	S3 (media), EFS (POS receipts)
Monitoring	CloudWatch, X-Ray, AWS Config

✓ Best Practices Summary

Principle	Recommendation
Design for Failure	Expect and simulate failure (e.g., chaos testing)
Decouple Everything	Use queues, streams, and async handlers
Auto Scale	Use ASG, Lambda, ECS with dynamic capacity
Use Managed Services	Prefer DynamoDB, S3, Lambda where possible
Cache Strategically	Use CloudFront + Redis to reduce latency/load
Multi-AZ Always	Avoid single points of failure
Be Idempotent	Especially for retries and APIs
Plan for DR	Choose a recovery strategy that fits your risk



6. Data Storage & Database Selection in AWS

Data is not one-size-fits-all. AWS offers a rich portfolio of storage and database options that serve different needs — from object stores and relational databases to high-performance NoSQL and massive-scale analytics engines.

In this section, we'll break down **how to choose, design, and implement** storage and database services tailored to your system design.



A. AWS Storage Options – The 3 Core Models

Storage Type	AWS Service	Best For
Object	S3	Static assets, backups, media, logs
Block	EBS	EC2 file systems, databases
File	EFS, FSx	Shared access file storage, lift-and-shift

◆ 1. Amazon S3 (Object Storage)

- Infinite durability (11 nines)
- Versioning, lifecycle rules, event notifications
- Supports static website hosting



CLI: Upload a file

```
aws s3 cp index.html s3://my-bucket-name/
```



Terraform: Create a public S3 bucket

```
resource "aws_s3_bucket" "static_site" {
```

Written by: MANISH KUMAR

```
bucket = "my-static-site"

website {
    index_document = "index.html"
    error_document = "error.html"
}

acl    = "public-read"
}
```

◆ 2. Amazon EBS (Block Storage)

- Used with EC2
- Persistent storage for operating systems, DBs
- Can be encrypted, snapshotted, backed up

CLI: Create encrypted EBS volume

```
aws ec2 create-volume \
    --size 20 \
    --availability-zone us-east-1a \
    --volume-type gp3 \
    --encrypted
```

◆ 3. Amazon EFS (File Storage)

- NFS-based, scalable file system
- Mount across EC2 instances in multiple AZs

CLI: Create EFS + Mount Targets

```
aws efs create-file-system --performance-mode generalPurpose
```

✓ Use EFS for apps that need shared file access like CMS, legacy systems, or training ML models on shared datasets.

B. AWS Database Models

AWS provides **purpose-built databases** for different use cases.

Database Type	AWS Service	Use Case
Relational (SQL)	RDS, Aurora	OLTP apps, enterprise systems
Key-Value	DynamoDB	High-speed NoSQL, serverless backends
Document	DocumentDB	JSON-based, MongoDB workloads
Graph	Neptune	Relationship mapping, social graphs
Time Series	Timestream	IoT, monitoring logs
Ledger	QLDB	Immutable logs, audit trails
In-memory	ElastiCache	Caching, real-time scoring

C. Amazon RDS (Relational DB Service)

Supports MySQL, PostgreSQL, SQL Server, Oracle.

Key Features:

- Automated backups
- Multi-AZ failover
- Read replicas
- Encryption, IAM auth

Terraform: Create a PostgreSQL DB

```
resource "aws_db_instance" "postgres" {
  identifier      = "my-postgres-db"
  engine          = "postgres"
  instance_class  = "db.t3.micro"
  username        = "admin"
```

```
password          = "securePass123"
allocated_storage = 20
skip_final_snapshot = true
}
```

 Use **Aurora** for auto-scaling, high-performance relational workloads.

D. Amazon DynamoDB (NoSQL, Key-Value/Document)

- Fully managed, serverless
- Single-digit ms response
- Global tables for multi-region sync

Use cases:

- User sessions, IoT data, gaming profiles, shopping carts

CLI: Create table

```
aws dynamodb create-table \  
  --table-name Users \  
  --attribute-definitions AttributeName=userId,AttributeType=S \  
  --key-schema AttributeName=userId,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST
```

IAM Policy to access DynamoDB

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Effect": "Allow",  
    "Action": "dynamodb:*",  
    "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Users"  
  }]  
}
```

💡 Combine with Lambda and API Gateway for full serverless APIs.

E. Global and Multi-Region Data

Options:

- **DynamoDB Global Tables** for real-time multi-region sync
- **S3 Cross-Region Replication** for backups or DR
- **Aurora Global Databases** for low-latency multi-continent reads

S3 CRR: Replicate objects to another region

```
aws s3api put-bucket-replication \  
  --bucket my-source-bucket \  
  --replication-configuration file://replication.json
```

F. Backup, Retention, and Archiving

Data architecture isn't complete without **retention and recovery plans**.

Tools:

- **AWS Backup** – Policy-based backups for EFS, RDS, DynamoDB, etc.
- **RDS Snapshots**
- **S3 Glacier** – For long-term cold storage

CLI: Move S3 files to Glacier

```
aws s3 cp myfile.txt s3://my-archive-bucket/ --storage-class GLACIER
```

G. Analytics-Ready Storage

Written by: MANISH KUMAR

For OLAP (analytical workloads), consider:

- **Redshift** – Petabyte-scale data warehouse
- **Athena** – Query S3 files using SQL (pay-per-query)
- **Lake Formation** – Secure, governed data lake
- **Glue** – Serverless ETL

Athena Example:

```
SELECT * FROM s3object s WHERE s.region = 'us-east-1';
```



H. Data Access & Security Considerations

Area	Best Practice
IAM	Use fine-grained policies
Encryption	Enable at rest (KMS) + in transit (TLS)
Tagging	Use tags for ownership, environment, and cost
Audit Trails	Enable CloudTrail on data services



I. Real-World Storage Strategy Example

Component	Service	Rationale
Static assets	Amazon S3 + CloudFront	Fast global delivery, versioning
App state	DynamoDB + DAX	Fast, scalable NoSQL
User data	RDS (Aurora)	Relational consistency needed
Analytics logs	S3 + Athena	Serverless querying, lake analytics
Cache	ElastiCache (Redis)	Reduce DB load
Archive	S3 Glacier Deep Archive	Compliance retention at lowest cost

Best Practices Summary

Category	Recommendation
Storage	Choose type: object (S3), block (EBS), file (EFS)
RDS	Use Multi-AZ + Read Replicas, encrypt by default
DynamoDB	Use on-demand mode for variable workloads
Backup	Use AWS Backup + S3 versioning + Glacier
Performance	Use ElastiCache, DAX, Aurora Serverless v2
Security	Encrypt all layers, apply IAM boundaries
Analytics	Combine S3 + Athena + Glue for data lake architecture



7. Networking & Connectivity Design in AWS

Designing a well-architected AWS network means **balancing security, performance, scalability, and cost**, all while ensuring **your services can talk to each other and to the outside world — securely and predictably**.

This section will cover foundational and advanced networking topics with hands-on CLI and IaC (Terraform) examples.



A. Virtual Private Cloud (VPC) – The Networking Foundation

A VPC is your **isolated slice of AWS cloud** where you define your networking environment:

- IP ranges (CIDR blocks)
- Subnets (public/private)
- Route tables
- NAT gateways
- Internet gateways
- Security groups & NACLs



VPC Structure Example:

```
VPC (10.0.0.0/16)
├── Public Subnet (10.0.1.0/24) ─> Internet Gateway (IGW)
│   └── EC2 Bastion, ALB
├── Private Subnet (10.0.2.0/24) ─> NAT Gateway
│   └── App Tier EC2, ECS, Lambda
└── DB Subnet (10.0.3.0/24) ─> No Internet
    └── RDS, ElastiCache
```

Terraform: Create a basic VPC

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id      = aws_vpc.main.id
  cidr_block  = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone = "us-east-1a"
}
```

CLI: Create an internet gateway

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-1234abcd --internet-gateway-id
igw-5678efgh
```

B. Network Security – SGs and NACLs

Feature	Security Group (SG)	Network ACL (NACL)
Type	Instance-level firewall	Subnet-level firewall
Stateful	✅ Yes	❌ No
Rule Direction	Inbound & Outbound	Inbound & Outbound
Rule Evaluation	All rules are evaluated	First rule match

✅ Best Practice: Use SGs for app-layer logic and NACLs for coarse-grained subnet protections.

C. Internet Access – IGW and NAT

- **Internet Gateway (IGW):** Enables internet access for public subnets.

- **NAT Gateway:** Allows private subnets to **initiate** internet access (e.g., for patching).

NAT Gateway Creation (CLI)

```
aws ec2 create-nat-gateway --subnet-id subnet-abc123 --allocation-id  
eipalloc-456def
```

- 💡 **NAT Instance** is cheaper but less scalable than **NAT Gateway**.

D. Internal Connectivity – VPC Peering, Transit Gateway

1. VPC Peering

- Direct connection between two VPCs.
- Transitive peering **not allowed**.

CLI: VPC Peering Request

```
aws ec2 create-vpc-peering-connection \  
--vpc-id vpc-aaa111 \  
--peer-vpc-id vpc-bbb222
```

2. Transit Gateway (TGW)

- Hub-and-spoke model to connect **hundreds of VPCs and on-premises**.
- Supports **multicast, route propagation, and centralized control**.

- ✅ Use TGW for large orgs or multi-region networks.

E. Hybrid Connectivity – VPN, Direct Connect, SD-WAN

Method	Description	Best For
Site-to-Site VPN	IPSec tunnel over internet	Quick hybrid cloud setup
AWS Direct Connect	Private leased line to AWS	High-throughput, consistent latency
SD-WAN	Vendor-managed network overlay	Scalable multi-site connectivity

CLI: Create VPN connection

```
aws ec2 create-vpn-connection \
  --type ipsec.1 \
  --customer-gateway-id cgw-123456 \
  --vpn-gateway-id vgw-abcdef
```

F. Route 53 – DNS & Routing Control

Use Route 53 for:

- Public & private DNS
- Latency/geolocation-based routing
- Health checks and failover

CLI: Create a public DNS zone

```
aws route53 create-hosted-zone \
  --name example.com \
  --caller-reference "unique-string"
```

 Use **failover routing policy** for DR and **alias records** for internal AWS targets.



G. PrivateLink, VPC Endpoints, and Service Access

Feature	Purpose
Interface Endpoint	Private IP for AWS services via ENI
Gateway Endpoint	Route table entry to S3 or DynamoDB over private
PrivateLink	Publish your own service privately in other VPCs



Create Gateway Endpoint to access S3 privately

```
aws ec2 create-vpc-endpoint \  
  --vpc-id vpc-1234abcd \  
  --service-name com.amazonaws.us-east-1.s3 \  
  --route-table-ids rtb-5678efgh \  
  --vpc-endpoint-type Gateway
```



VPC Endpoints eliminate NAT costs and enhance security.



H. Monitoring, Logging, and Traffic Flow

Tool	Purpose
VPC Flow Logs	Record IP traffic metadata
CloudWatch	Alarms, metrics, dashboards
CloudTrail	API-level audit trails
Reachability Analyzer	Simulate traffic flow



Enable Flow Logs for a VPC

```
aws ec2 create-flow-logs \  
  --resource-type VPC \  
  --resource-ids vpc-1234abcd \  
  --traffic-type ALL \  
  --log-group-name vpc-logs \  
  --deliver-logs-permission-arn arn:aws:iam::123456789012:role/FlowLogsRole
```



I. Sample Architecture – Secure Public Web App

Component	AWS Service(s)	Role
Frontend	ALB in public subnet + WAF	Entry point, load balancing
App Layer	ECS Fargate in private subnet	Business logic
DB Layer	RDS in isolated subnet	Persistent storage
Internet Access	IGW (frontend), NAT (backend)	Controlled connectivity
DNS	Route 53 + ALIAS to ALB	Domain routing
Monitoring	CloudWatch + Flow Logs + X-Ray	Observability



Best Practices Summary

Area	Recommendation
Subnet Planning	Use separate subnets for public, private, DB
NAT Optimization	Use NAT Gateway in one AZ with failover if needed
DNS Design	Use Route 53 for DNS + health checks
Service Access	Use VPC Endpoints for secure access to AWS services
Hybrid Setup	Prefer Direct Connect for prod, VPN for quick tests
Central Routing	Use TGW in multi-account/multi-VPC designs
Security Controls	Enforce strict SG + NACLs, use PrivateLink
Monitoring	Enable VPC Flow Logs, CloudWatch alarms, AWS Config



8. Security, Identity & Compliance in AWS

This section covers the **AWS security model**, **identity and access management**, **encryption**, **secrets handling**, **monitoring**, and **governance** — all illustrated with real-world configurations and automation examples.



A. The Shared Responsibility Model

Responsibility Type	AWS (Cloud Provider)	You (Customer)
Security <i>of</i> the Cloud	Hardware, global infra, managed services	✗ Not your job
Security <i>in</i> the Cloud	Identity, network, OS, apps, data	✓ Your responsibility



Example: AWS secures S3 infra, **you must secure S3 buckets**.



B. IAM – Identity & Access Management

IAM governs **who** can do **what** on **which** resources.

Key Concepts:

- **Principals:** Users, roles, federated identities
- **Policies:** JSON-based permission documents
- **Roles:** Assumed by services (like Lambda, EC2, etc.)
- **Temporary credentials** via STS
- **IAM Conditions:** Time, IP, MFA, resource tags



CLI: Create an IAM user with programmatic access

```
aws iam create-user --user-name devuser
```

```
aws iam create-access-key --user-name devuser
```

Terraform: Create IAM policy + role

```
resource "aws_iam_role" "lambda_exec" {
  name = "lambda_execution_role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Action = "sts:AssumeRole",
      Principal = { Service = "lambda.amazonaws.com" },
      Effect = "Allow",
    }]
  })
}

resource "aws_iam_policy" "lambda_policy" {
  name          = "lambda_logging_policy"
  description   = "Allow Lambda to log to CloudWatch"
  policy        = file("lambda_policy.json")
}

resource "aws_iam_role_policy_attachment" "lambda_attach" {
  role          = aws_iam_role.lambda_exec.name
  policy_arn    = aws_iam_policy.lambda_policy.arn
}
```

C. Encryption – At Rest and In Transit

Scope	AWS Tool	Best Practice
At Rest	KMS, S3, RDS, EBS	Enable encryption everywhere
In Transit	TLS/SSL	Enforce HTTPS/SSL only
Custom Keys	AWS KMS	Use CMKs + rotation policies
Client-side	SDKs, Envelope Encryption	For ultra-sensitive apps

CLI: Create a KMS CMK

```
aws kms create-key --description "App encryption key"
```

 You can use KMS CMK ARNs in services like S3, RDS, EBS for automatic encryption.

D. Secrets Management

Storing credentials or API keys in code is a **fatal security flaw**.

Options:

Service	Best For
AWS Secrets Manager	DB passwords, API keys (rotatable)
AWS Systems Manager Parameter Store	Config values, feature flags

CLI: Store secret in Secrets Manager

```
aws secretsmanager create-secret \  
  --name MyAppDBPassword \  
  --secret-string '{"username":"admin","password":"SecurePass123"}'
```

Terraform: Create secret with rotation

```
resource "aws_secretsmanager_secret" "db_password" {  
  name = "db-password"  
  rotation_lambda_arn = aws_lambda_function.rotate_secret.arn  
  rotation_rules {  
    automatically_after_days = 30  
  }  
}
```

E. Network Security – SGs, NACLs, WAF, Shield, Firewall Manager

Written by: MANISH KUMAR

Control	Purpose
SGs & NACLs	Instance & subnet firewalls
AWS WAF	Layer 7 protection from bad traffic
AWS Shield	DDoS protection
Firewall Manager	Central policy management for org

CLI: Deploy AWS WAF Web ACL with IP block

```
aws wafv2 create-web-acl \
  --name block-bots \
  --scope REGIONAL \
  --default-action Allow={} \
  --rules file://rules.json
```

F. Identity Federation – SSO & External IdP

Federate access from:

- **SAML (Okta, Azure AD, etc.)**
- **OIDC (Google, GitHub, Cognito)**

Use cases:

- Workforce access to AWS Console
- Federated login into web/mobile apps

Terraform: AWS IAM Identity Provider

```
resource "aws_iam_openid_connect_provider" "github" {
  url = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = ["9e99a48a9960b14926bb7f3b02e22da0afd10df6"]
}
```



G. Compliance & Governance – Guardrails

Tool	Description
AWS Config	Record resource state changes
AWS Organizations	Structure multi-account with SCPs
Service Control Policies (SCP)	Restrict services/accounts globally
IAM Access Analyzer	Detect unintended public/shared access
AWS Audit Manager	Generate compliance reports (SOC2, PCI, etc.)

SCP Example: Deny All Except EC2 + S3

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "NotAction": [
      "ec2:*",
      "s3:*"
    ],
    "Resource": "*"
  }]
}
```



H. Security Monitoring Tools

Tool	Function
CloudTrail	Audit API activity
Security Hub	Central view of findings
Inspector	Scan for vulnerabilities on EC2/ECR
Macie	PII detection in S3
GuardDuty	Threat detection + alerting
Detective	Forensic investigation post-incident

 Connect **Security Hub** → **GuardDuty** → **SNS** for automated alerts.

I. Real-World Use Case: Secure Microservice

Architecture

Layer	Control Applied
Public API Gateway	AWS WAF + Rate Limiting
Private Services	IAM + SG-based access control
DB Access	Use IAM auth (RDS/Dynamo) + Secrets Manager
Cross-Account	IAM roles + Resource-based policies
User Login	Cognito Federated Identity + Custom Domain
Monitoring	CloudTrail + Config + Security Hub + GuardDuty
Backup Security	Enable encryption + restrict IAM backup access

Best Practices Summary

Area	Best Practice
IAM	Use least privilege; separate roles for apps and users
Secrets	NEVER store in code; use Secrets Manager or SSM
Encryption	Enable KMS across all services; use custom keys if needed
SCPs	Use deny-by-default for critical accounts
WAF & Shield	Protect web apps with regional ACLs + L7 filtering
Audit & Monitoring	Centralize with CloudTrail, Config, and Security Hub
IAM Analyzer	Detect unintended public exposure
Compliance	Use AWS Audit Manager for continuous assessment



9. Observability & Monitoring

Architecture in AWS

Observability is more than just logging — it's about **understanding the internal state of your system through its outputs**: metrics, logs, traces, alerts, dashboards, and user experience signals. A well-designed monitoring strategy enables:

- Faster incident detection
- Root-cause isolation
- Performance optimization
- Capacity planning
- Compliance & audit readiness



A. The AWS Observability Stack

Type	AWS Service(s)	Purpose
Metrics	CloudWatch Metrics, Container Insights	Track performance and usage
Logs	CloudWatch Logs, S3, Firehose	Store and analyze events
Tracing	AWS X-Ray, OpenTelemetry	Visualize end-to-end request path
Dashboards	CloudWatch Dashboards	Real-time monitoring views
Alarms	CloudWatch Alarms, EventBridge	Alerting based on thresholds



B. CloudWatch – Metrics, Alarms, Dashboards



1. Metrics

You get **default metrics** for AWS services (CPU, memory, I/O) and can **push custom metrics** via the SDK or CLI.

CLI: Push a custom metric

```
aws cloudwatch put-metric-data \  
  --namespace "MyApp" \  
  --metric-name "OrdersProcessed" \  
  --value 57 \  
  --unit Count
```

 You can also use the CloudWatch Agent or CloudWatch Embedded Metric Format (EMF) in logs.

2. Alarms

Create alarms for anomalies, thresholds, or custom metrics, with alerting via SNS, EventBridge, or auto-remediation.

Terraform: CPU Alarm with SNS Notification

```
resource "aws_cloudwatch_metric_alarm" "high_cpu" {  
  alarm_name      = "HighCPUALarm"  
  comparison_operator = "GreaterThanThreshold"  
  evaluation_periods = 2  
  metric_name     = "CPUUtilization"  
  namespace      = "AWS/EC2"  
  period         = 60  
  statistic       = "Average"  
  threshold       = 75  
  alarm_description = "EC2 CPU > 75%"  
  alarm_actions    = [aws_sns_topic.alerts.arn]  
}
```

3. Dashboards

Visualize metrics and logs using **CloudWatch Dashboards** for operations and business KPIs.

```
aws cloudwatch put-dashboard \  
  --dashboard-name AppHealth \  
  --dashboard-body file://dashboard.json
```

C. CloudWatch Logs – Centralized Logging

Log aggregation is essential for troubleshooting, auditing, and compliance.

◆ Integrate Logs From:

- EC2 via **CloudWatch Agent**
- Lambda (automatic)
- ECS/Fargate via FireLens
- EKS via FluentBit + OpenTelemetry
- VPC Flow Logs, Route 53 logs, Load Balancer logs

CLI: Create log group and stream

```
aws logs create-log-group --log-group-name /app/orders  
aws logs create-log-stream --log-group-name /app/orders --log-stream-name  
prod
```

Terraform: CloudWatch Agent Configuration

```
resource "aws_ssm_parameter" "cwagent_config" {  
  name = "/AmazonCloudWatch/linux-config"  
  type = "String"  
  value = file("cwagent.json")  
}
```

D. Tracing with AWS X-Ray

X-Ray provides **distributed tracing**, showing how requests propagate across microservices, Lambdas, databases, etc.

Feature	Benefit
Trace Maps	Visual path of a request
Annotations	Custom key-value pairs
Sampling Rules	Control cost and coverage

SDK: Add X-Ray to a Python Lambda

```
from aws_xray_sdk.core import patch_all, xray_recorder
patch_all()

def lambda_handler(event, context):
    with xray_recorder.in_subsegment("db_query"):
        data = query_db()
```

✅ Integrate with App Mesh, ALB, ECS, Lambda, API Gateway, and custom apps via SDK.

E. Container Observability (ECS/EKS)

ECS (Fargate or EC2):

- **CloudWatch Container Insights** for CPU/memory
- **FireLens** with FluentBit to ship logs to CW/S3/3rd-party

```
"logConfiguration": {
  "logDriver": "awsfirelens",
  "options": {
    "Name": "cloudwatch",
    "region": "us-east-1",
    "log_group_name": "/ecs/app",
    "auto_create_group": "true"
  }
}
```

EKS:

- **Prometheus + Grafana + OpenTelemetry** for metrics
- **FluentBit or Fluentd** for logs to CloudWatch
- **AWS Distro for OpenTelemetry (ADOT)**

F. Global Event Monitoring – EventBridge

Use **EventBridge** to route events across AWS services, automate responses, or trigger workflows.

Example:

- Lambda fails → EventBridge rule → notify via SNS & trigger rollback

```
{  
  "source": ["aws.lambda"],  
  "detail-type": ["Lambda Function Invocation Result - Failure"]  
}
```

G. Cost-Aware Monitoring

Service	Metric	Use Case
AWS Budgets	Actual vs forecast spend	Alerts on overspend
Cost Explorer API	Usage trends, resource drill-down	Optimization analysis
CloudWatch Logs	Query patterns to find inefficiencies	E.g., Lambda duration spikes



H. Use Case Example: Production-Grade Observability Stack

Layer	Tool/Service	Notes
Metrics	CloudWatch + Container Insights	Unified metrics across stack
Logs	CloudWatch Logs + S3 Archive	Retention + compliance
Tracing	X-Ray or OpenTelemetry Collector	Microservices performance
Dashboards	CloudWatch + Grafana via AMP	Operational & business KPIs
Alerts	CloudWatch Alarms + EventBridge + SNS	Multi-channel notifications
Automation	Lambda responders / rollback triggers	Auto-remediation



Best Practices Summary

Domain	Best Practice
Metrics	Use custom namespaces for app-level metrics
Logging	Use structured logs (JSON) with unique request IDs
Tracing	Use X-Ray or OpenTelemetry; trace all major services
Dashboards	Group by team/service; display top KPIs + error trends
Alerts	Use severity levels (warn/error/critical) + auto-actions
Cost Optimization	Use sampling for X-Ray and logs; archive older logs to S3
Governance	Tag log groups and dashboards by project/team for clarity



10. Designing for Resilience, Availability & Fault Tolerance

In cloud-native design, failure is **inevitable**. The key is not to avoid failure, but to **gracefully tolerate and recover from it**. This section covers principles and patterns used by AWS solutions architects to build systems that remain available and consistent **even in the face of disruption**.



A. Core Principles

Concept	Meaning
High Availability	System remains accessible (e.g., via HAProxy, load balancing, failover)
Fault Tolerance	System continues functioning despite component failures
Resilience	System recovers quickly from disruptions
Durability	Data remains intact despite failures



B. Multi-AZ & Multi-Region Architecture

◆ Multi-AZ:

- Deploy across **Availability Zones** (isolated data centers in the same region)
- Automatic failover with services like:
 - RDS Multi-AZ
 - ECS with ALB spanning AZs
 - EC2 Auto Scaling Groups (ASGs)

◆ Multi-Region:

- For disaster recovery, global apps, and geo-distributed users
- Examples:
 - Active-Passive (e.g., Route 53 failover routing)
 - Active-Active (e.g., DynamoDB Global Tables, S3)

Example: Multi-AZ Load Balanced Web App

1. Terraform: EC2 ASG across AZs + ALB

```
resource "aws_autoscaling_group" "web" {
  desired_capacity = 3
  max_size        = 6
  min_size        = 2
  vpc_zone_identifier = [aws_subnet.az1.id, aws_subnet.az2.id]
  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
  target_group_arns = [aws_lb_target_group.web.arn]
}
```

2. Route53 Health-Check Based Failover

```
resource "aws_route53_record" "app_failover" {
  zone_id = aws_route53_zone.primary.zone_id
  name     = "app.example.com"
  type     = "A"
  set_identifier = "primary"
  failover    = "PRIMARY"
  health_check_id = aws_route53_health_check.app.id
  alias {
    name           = aws_lb.primary.dns_name
    zone_id        = aws_lb.primary.zone_id
    evaluate_target_health = true
  }
}
```

C. Key AWS Services Supporting Resilience

Service	Resilience Feature
S3	99.999999999% durability, versioning, replication
RDS	Multi-AZ deployment, read replicas, backups
EC2	Auto Recovery, ASG, elastic IP, instance refresh
EFS	Regional, multi-AZ NFS file system
DynamoDB	Multi-AZ and Global Tables
SQS + SNS	Decoupling, retry, dead-letter queues
Lambda	Multi-AZ deployment, retry on failure

D. Self-Healing Infrastructure

Use **automation** to replace unhealthy components without manual intervention.

1. EC2 Auto Recovery with CloudWatch Alarm

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "RecoverEC2Instance" \  
  --metric-name StatusCheckFailed_System \  
  --namespace AWS/EC2 \  
  --statistic Average \  
  --period 60 \  
  --threshold 1 \  
  --comparison-operator GreaterThanOrEqualToThreshold \  
  --dimensions Name=InstanceId,Value=i-1234567890abcdef0 \  
  --evaluation-periods 2 \  
  --alarm-actions arn:aws:automate:us-east-1:ec2:recover
```

2. ASG with Instance Health Check

```
resource "aws_autoscaling_group" "example" {  
  health_check_type = "EC2"  
  health_check_grace_period = 300
```

}

E. Backup & Recovery Strategy

Design for **data durability and recovery objectives**:

Objective	Meaning	Tool Examples
RPO (Recovery Point)	Max data loss allowed	AWS Backup, snapshots, S3 versioning
RTO (Recovery Time)	Max time to recover a system	CloudEndure, EC2 AMIs, cross-region DR

Terraform: Backup Vault + Backup Plan

```
resource "aws_backup_vault" "main" {
  name = "project-backup-vault"
}

resource "aws_backup_plan" "default" {
  name = "daily-backup-plan"
  rule {
    rule_name      = "daily-backup"
    target_vault_name = aws_backup_vault.main.name
    schedule       = "cron(0 5 * * ? *)"
    lifecycle {
      delete_after = 90
    }
  }
}
```

F. Designing for Graceful Degradation

Your system should continue offering partial functionality even if some components fail.

Example: If recommendation service fails, show default products instead of crashing UI.

Patterns:

- Use **circuit breakers** (e.g., in AWS AppConfig)
- Provide **fallback responses**
- Queue failed requests (SQS DLQ)



G. Disaster Recovery Strategies

Strategy	Cost	RTO/RPO	Description
Backup & Restore	\$	Hours	Copy from S3 or AMIs
Pilot Light	\$\$	Minutes	Pre-provisioned minimal environment
Warm Standby	\$\$\$	Seconds-Min	Scaled-down clone of production
Multi-Site Active-Active	\$\$\$\$	<Seconds	Global traffic routing, fully active setups



Route53 + Global Accelerator + CloudFront = Global Active-Active setup



Best Practices Summary

Principle	Best Practice
AZ Redundancy	Always span multiple Availability Zones
Auto Scaling	Use ASGs for web/app tiers with health checks
Data Durability	Enable S3 versioning, use DynamoDB TTL, backup with AWS Backup
Circuit Breakers	Gracefully handle failure (AppConfig, Lambda fallback, Step Function retries)
Cross-Region DR	Use S3 CRR, DynamoDB Global Tables, RDS read replica in another region
Health Monitoring	CloudWatch alarms + ASG health checks + Route 53 failover routing
Event-Driven Healing	EventBridge + Lambda for remediation



Bonus Tools for Resilience Testing

Tool	Use Case
AWS Fault Injection Simulator (FIS)	Controlled chaos engineering
AWS Resilience Hub	Score + improve architecture resilience
AWS Well-Architected Tool	Review and recommendations



11. Designing Cost-Optimized Architectures in AWS

AWS gives you a **plethora of pricing models**, resources, and services. The key to cost optimization is understanding **how resources are billed**, where **waste occurs**, and how to use **automation and monitoring to reduce spend without compromising performance**.



A. Core Principles of Cost Optimization

Principle	Explanation
Right-sizing	Match instance type and size to actual usage
Elasticity	Scale up/down automatically based on demand
Pricing Model Selection	Choose On-Demand, Spot, Reserved Instances wisely
Monitoring & Alerts	Use Budgets, Cost Explorer, and CloudWatch to monitor and act
Avoid Orphaned Resources	Clean unused volumes, snapshots, IPs, ELBs, etc.
Storage Lifecycle	Use tiered storage and archival (e.g., S3 IA, Glacier)



B. EC2 & Compute Optimization

◆ Instance Selection

Use **Graviton-based instances** (e.g., **t4g**, **m6g**) for 20-40% cost savings.

◆ Right-Sizing with Compute Optimizer

AWS Compute Optimizer uses ML to recommend better instance types based on usage.

```
aws compute-optimizer get-ec2-instance-recommendations \
```

```
--instance-arns arn:aws:ec2:us-east-1:123456789012:instance/i-abc123
```

◆ Spot Instances for Flexible Workloads

```
resource "aws_launch_template" "spot" {  
  instance_market_options {  
    market_type = "spot"  
    spot_options {  
      max_price = "0.02"  
    }  
  }  
}
```

Use for:

- CI/CD jobs
- Batch processing
- Container nodes (EKS)

C. Storage Optimization

◆ S3 Storage Classes

Class	Use Case	Cost
Standard	Frequently accessed data	High
Intelligent-Tiering	Varies usage (auto-saves cost)	Medium
S3 IA / One Zone IA	Archive, infrequent access	Low
Glacier / Deep Archive	Compliance, cold archive	Lowest

Lifecycle Policy (S3 → Glacier)

```
{  
  "Rules": [  
    {  
      "ID": "MoveToGlacier",
```

```

    "Status": "Enabled",
    "Prefix": "",
    "Transitions": [
      {
        "Days": 30,
        "StorageClass": "GLACIER"
      }
    ]
  }
]
}

```

◆ EBS Volume Rightsizing

1. Identify idle volumes with **CloudWatch VolumeIdleTime**
2. Snapshot and delete unused EBS volumes.

```
aws ec2 describe-volumes --filters Name=status,Values=available
```

D. Serverless Cost Control

◆ Lambda

- Cost = # of invocations + GB-seconds
- Optimize memory config with **power tuning tools**
- Avoid idle functions polling — use **EventBridge, SQS, Step Functions**

◆ Example: Lambda Cost Estimation

```
aws lambda get-function-url-config --function-name MyFunction
```

- If using 512MB for 2 million monthly requests with 1s duration:

Cost = ~\$12/month

E. Database Cost Optimization

◆ RDS

- Use **RDS Reserved Instances (RI)** or **Aurora Serverless v2** for variable workloads.
- Enable **auto-pause** in dev/test environments.
- Enable **storage autoscaling**.

◆ DynamoDB

- Use **On-Demand mode** for spiky traffic
- Use **provisioned + auto-scaling** for steady load
- Enable **DAX caching** to reduce read cost

```
aws dynamodb update-table \  
  --table-name MyTable \  
  --billing-mode PAY_PER_REQUEST
```

F. Networking Optimization

◆ Avoid NAT Gateway Overuse

NAT Gateway = **\$0.045/hr** + **\$0.045/GB**

For large data egress, prefer:

- **VPC Endpoints** for S3/DynamoDB (no NAT usage)
- **PrivateLink** for private service access

```
resource "aws_vpc_endpoint" "s3" {  
  service_name = "com.amazonaws.us-east-1.s3"  
  vpc_id       = aws_vpc.main.id  
}
```



G. Monitoring & Budgeting Tools

Tool	Use For
AWS Budgets	Set alerts for actual or forecasted cost
Cost Explorer	Visualize usage patterns
Cost Anomaly Detection	Detect and alert on spend anomalies
CloudWatch Dashboards	Track cost metrics and alerts



Create Budget

```
aws budgets create-budget \  
  --account-id 123456789012 \  
  --budget file://budget.json
```

Sample JSON:

```
{  
  "BudgetName": "MonthlyLimit",  
  "BudgetLimit": { "Amount": "100", "Unit": "USD" },  
  "TimeUnit": "MONTHLY",  
  "BudgetType": "COST",  
  "CostFilters": { "Service": ["AmazonEC2"] }  
}
```



H. Containers Cost Optimization (ECS, EKS)

- **Use Fargate Spot** (up to 70% cheaper than On-Demand)
- **Bin packing:** Use tools like **Karpenter** (EKS) or cluster autoscaler to reduce idle capacity
- **Enable horizontal/vertical pod autoscaling**

```
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
metadata:
```

```

    name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

```



I. Scheduled Scaling & Shutdowns

For dev/test environments, use **Instance Scheduler** or Lambda cron to stop/start resources.



Scheduled Auto Scaling Policy

```

resource "aws_autoscaling_schedule" "scale_in" {
  scheduled_action_name = "scale-in-night"
  min_size              = 0
  max_size              = 0
  desired_capacity      = 0
  recurrence             = "0 22 * * *"
  autoscaling_group_name = aws_autoscaling_group.web.name
}

```

Best Practices Summary

Category	Best Practice
Compute	Use Graviton, Spot, Auto Scaling, Lambda
Storage	Use S3 tiering, delete old snapshots
DB	On-Demand mode for DynamoDB, Serverless Aurora
Monitoring	Budgets, anomaly detection, CloudWatch insights
Networking	Use VPC endpoints, avoid NAT gateway overuse
Dev/Test	Use scheduling to shut down off-hours resources
Containers	Bin-packing, Fargate Spot, HPA/VPA/Karpenter



12. Security, Compliance & Identity in AWS Architecture

In AWS, **security is a shared responsibility**: AWS secures the infrastructure, and **you secure what you build on top** — your applications, data, IAM policies, and network configurations.

This section outlines how to architect **secure systems**, meet **compliance standards**, and use **identity services and encryption tools** effectively.



A. AWS Shared Responsibility Model

Layer	AWS Responsibility	Your Responsibility
Infrastructure	Physical security, network, hypervisor	OS, application, firewall settings
Services (PaaS/SaaS)	Underlying platform (e.g. Lambda infra)	IAM policies, secure code, data encryption
Data	—	Data integrity, encryption, classification



B. Identity & Access Management (IAM)

◆ Core Best Practices

- **Least Privilege Principle** — Start with deny, then allow only necessary actions.
- **IAM Roles > IAM Users** — Avoid long-term credentials.
- **Use Permissions Boundaries** — Limit delegated role capabilities.



IAM Role for EC2 to Access S3 (Terraform)

```
resource "aws_iam_role" "ec2_role" {
```

Written by: MANISH KUMAR

```

name = "ec2_s3_access_role"
assume_role_policy = jsonencode({
  Version = "2012-10-17",
  Statement = [{
    Action = "sts:AssumeRole",
    Effect = "Allow",
    Principal = {
      Service = "ec2.amazonaws.com"
    }
  }]
})
}

resource "aws_iam_policy" "s3_access" {
  name = "s3-read-policy"
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [{
      Effect = "Allow",
      Action = ["s3:GetObject"],
      Resource = "arn:aws:s3:::mybucket/*"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "attach" {
  role      = aws_iam_role.ec2_role.name
  policy_arn = aws_iam_policy.s3_access.arn
}

```

C. Data Protection & Encryption

◆ At Rest

Service	Encryption Option
S3	SSE-S3, SSE-KMS, SSE-C
EBS	EBS volume encryption with KMS
RDS	Enable encryption during creation
DynamoDB	KMS-managed encryption (default)

```
aws kms create-key --description "EBS encryption key"
```

◆ In Transit

- Always enforce **HTTPS (TLS)** using CloudFront, ALB, API Gateway
- Use **SSL/TLS certificates** via ACM (free in AWS)

```
aws acm request-certificate \
  --domain-name "example.com" \
  --validation-method DNS
```

D. Network Security

◆ Security Groups vs. NACLs

Feature	Security Group	NACL
Stateful	✅ Yes	❌ No
Rules	Allow only	Allow + Deny
Scope	EC2 instances	Subnets

◆ Best Practices

- Use **private subnets** for databases and backend services
- Restrict **SSH/RDP** with bastion or SSM
- Use **VPC endpoints** for secure access to AWS services

E. WAF, Shield & GuardDuty

Tool	Purpose
AWS WAF	Protect web apps from common threats (SQLi, XSS)
Shield	DDoS protection
GuardDuty	Intelligent threat detection from logs & flows

◆ WAF Rule for Blocking SQL Injection

```
{
  "Name": "BlockSQLi",
  "Priority": 1,
  "Action": { "Block": {} },
  "Statement": {
    "ByteMatchStatement": {
      "FieldToMatch": { "QueryString": {} },
      "PositionalConstraint": "CONTAINS",
      "SearchString": "UNION SELECT",
      "TextTransformations": [{ "Type": "NONE", "Priority": 0 }]
    }
  }
}
```

F. Compliance & Governance

◆ Tools for Compliance

Tool	Use Case
AWS Config	Track resource compliance (auto snapshot)
AWS Organizations + SCPs	Enforce policy boundaries at account level
CloudTrail	Audit logs for all actions
AWS Audit Manager	Prebuilt frameworks (PCI, HIPAA, ISO)

SCP Example: Deny S3 Public Access Across Org

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "s3:PutBucketPolicy",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "s3:PublicAccessBlockConfiguration": "false"
      }
    }
  }]
}
```

G. Monitoring, Logging & Alerts

◆ CloudTrail

- Tracks **who did what** in your AWS account
- Store logs in S3 (and optionally encrypt)

```
aws cloudtrail create-trail \
  --name mytrail \
  --s3-bucket-name my-trail-bucket
```

◆ CloudWatch Logs + Alarms

- Real-time visibility into logs, metrics, custom alerts

```
aws cloudwatch put-metric-alarm \
  --alarm-name "HighCPU" \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --threshold 80 \
```

```
--comparison-operator GreaterThanThreshold \  
--evaluation-periods 2 \  
--statistic Average \  
--period 300 \  
--alarm-actions arn:aws:sns:us-east-1:123456789012:NotifyMe
```

✓ Best Practices Summary

Area	Best Practice
IAM	Enforce least privilege, use roles not users
Data Security	Use KMS, enable encryption at rest and in transit
Networking	Use SGs/NACLs properly, isolate VPC tiers
Monitoring	Enable CloudTrail, GuardDuty, and centralized logging
Compliance	Use AWS Config + SCPs to maintain standards
Threat Defense	WAF, Shield, and GuardDuty for application protection



13. Monitoring, Logging & Observability in AWS Architecture

Modern cloud systems are **distributed, elastic, and dynamic**. Without a well-planned observability strategy, your architecture becomes a black box. In this section, you'll learn how to **collect, analyze, visualize, and respond** to system behavior using AWS-native tools and best practices.



A. What is Observability?

Observability is more than just monitoring — it's about having **full visibility into internal system states** using:

- **Metrics** – Quantitative measurements (CPU usage, latency)
- **Logs** – System and application logs
- **Traces** – Distributed request path analysis (end-to-end)
- **Events** – Notifications or state changes (e.g., resource scaling)

Think: **metrics for alerts, logs for debugging, traces for root cause, and events for automation.**



B. CloudWatch: Metrics, Dashboards, and Alarms

◆ Metrics Collection

CloudWatch automatically collects default metrics for AWS services.

- **EC2:** `CPUUtilization`, `DiskReadOps`, `NetworkOut`
- **RDS:** `ReadIOPS`, `FreeStorageSpace`
- **Lambda:** `Duration`, `Invocations`, `Errors`

Custom Metrics (CLI)

```
aws cloudwatch put-metric-data \  
  --metric-name CustomAppLatency \  
  --namespace "MyApp" \  
  --value 350 \  
  --unit Milliseconds
```

Dashboards (Terraform)

```
resource "aws_cloudwatch_dashboard" "main" {  
  dashboard_name = "system-overview"  
  dashboard_body = jsonencode({  
    widgets = [{  
      type = "metric",  
      x = 0,  
      y = 0,  
      width = 12,  
      height = 6,  
      properties = {  
        metrics = [["AWS/EC2", "CPUUtilization", "InstanceId", "i-abc123"]],  
        title = "EC2 CPU"  
      }  
    }]  
  })  
}
```

Alarms Example

```
aws cloudwatch put-metric-alarm \  
  --alarm-name "LambdaErrorRate" \  
  --metric-name Errors \  
  --namespace AWS/Lambda \  
  --statistic Sum \  
  --period 60 \  
  --threshold 5 \  
  --comparison-operator GreaterThanThreshold \  
  --evaluation-periods 3
```

```
--evaluation-periods 1 \  
--alarm-actions arn:aws:sns:us-east-1:123456789012:NotifyTeam
```



C. Logging with CloudWatch Logs

◆ Log Sources

Source	Example Logs
EC2	Application, system logs
Lambda	stdout logs, execution traces
API Gateway	Request/response, latency
VPC Flow Logs	Network-level traffic logs
CloudTrail	Audit logs (who did what, where)



Setup (for EC2 Linux)

```
sudo yum install -y awslogs  
sudo vim /etc/awslogs/awslogs.conf
```

Configure log group and files:

```
[/var/log/messages]  
file = /var/log/messages  
log_group_name = ec2-system  
log_stream_name = {instance_id}
```

Start the agent:

```
sudo systemctl start awslogsd
```



D. Distributed Tracing with AWS X-Ray

X-Ray is used for **visualizing application request paths**, identifying latency bottlenecks, and root cause analysis.

Example: Tracing in Lambda (Node.js)

```
const AWSXRay = require('aws-xray-sdk');
const AWS = AWSXRay.captureAWS(require('aws-sdk'));

exports.handler = async (event) => {
  const s3 = new AWS.S3();
  await s3.listBuckets().promise();
  return { statusCode: 200 };
};
```

Enable tracing via console or CloudFormation:

```
TracingConfig:
  Mode: Active
```

X-Ray supports Lambda, ECS, EC2, API Gateway, Step Functions, etc.

E. Event-Driven Observability: EventBridge

◆ Example: Triggering Lambda on EC2 Stop

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": { "state": ["stopped"] }
}
```

EventBridge can trigger:

- Lambda functions
- SNS notifications
- Step Functions
- SQS queues

This is useful for **automated responses** to security changes, scaling, failures, etc.

F. Centralized Logging with CloudTrail & S3

◆ CloudTrail – Capture API Calls Across AWS

```
aws cloudtrail create-trail \  
  --name globalTrail \  
  --s3-bucket-name my-org-logs
```

Includes:

- Console & CLI/API activity
- Identity of caller
- Source IP
- Resource touched

Useful for compliance, debugging, and forensics.

G. Third-Party Integration

Tool	Use Case
Datadog	Full observability, alerting
Prometheus	Metrics scraping (EKS, Fargate)
Grafana	Dashboards for Prometheus/CW
New Relic	APM + custom dashboards
Fluent Bit	Log forwarding from containers

✓ Best Practices Summary

Area	Best Practice
Metrics	Use CloudWatch metrics + custom application metrics

Logs	Centralize using CloudWatch Logs + structured log formats
Tracing	Use X-Ray for distributed applications
Events	Use EventBridge for automation & system awareness
Dashboards	Build CloudWatch or Grafana dashboards by workload/service
Anomaly Detection	Enable CloudWatch Anomaly or use ML services (Lookout)
Retention	Set log retention policies to reduce cost

Example: Observability Stack for a Real-World Web App

Layer	Tool/Service
Metrics	CloudWatch + Prometheus
Logs	CloudWatch Logs + Fluent Bit
Traces	AWS X-Ray (Lambda/API Gateway/ECS)
Alerts	CloudWatch Alarms → SNS
Visualization	CloudWatch Dashboards + Grafana
Audit Trail	AWS CloudTrail + AWS Config + S3



15. Disaster Recovery (DR) & High Availability (HA) Strategies

Every business must prepare for the **unexpected** — whether it's hardware failure, a region-wide outage, a misconfiguration, or a natural disaster. As an AWS System Architect, your job is to **design for resilience, minimize downtime, and plan recovery** with clear RTO/RPO objectives.



A. Key Concepts

Term	Definition
High Availability (HA)	Designing systems to stay online during failures (AZ, node, etc.)
Fault Tolerance	The system keeps functioning <i>without</i> degradation when a failure occurs
Disaster Recovery (DR)	Preparing for region-wide or catastrophic failures with failover/recovery plans
RTO (Recovery Time Objective)	Maximum acceptable time to restore service
RPO (Recovery Point Objective)	Maximum acceptable data loss window



B. Availability Zones & Regions Strategy

◆ Availability Zones (AZs)

Each AWS region has **multiple AZs**. Architect to use at least **two AZs** for:

- Load-balanced apps
- HA RDS

- Multi-AZ ECS tasks

```
resource "aws_lb" "web" {
  name           = "app-lb"
  load_balancer_type = "application"
  subnets       = var.public_subnet_ids # spread across AZs
}
```

◆ Multi-Region Strategy

Use multi-region only when **business demands zero downtime** or you must meet **data sovereignty/compliance**.

Examples:

- Active-Active: Low RTO/RPO, global latency optimization
- Active-Passive: Warm/Cold failover

C. DR Patterns & Use Cases

DR Pattern	RTO	RPO	Cost	Description
Backup & Restore	Hours	Hours	Low	Store backups in S3, restore when needed
Pilot Light	Minutes	Low	Medium	Core services always running; scale out during failover
Warm Standby	<1 Min	Low	High	Scaled-down clone in standby
Multi-Site Active/Active	Seconds	Near zero	Very High	Live traffic across multiple regions

Example: Pilot Light for RDS + EC2

- Use **Amazon RDS cross-region snapshot copy**
- Sync EC2 AMIs and data to another region
- Use Route53 health checks + failover routing

```
# Copy RDS snapshot
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier rds-prod-snapshot \
  --target-db-snapshot-identifier dr-snapshot-copy \
  --source-region us-east-1 \
  --region us-west-2
# Route53 failover
aws route53 change-resource-record-sets \
  --hosted-zone-id ZONEID \
  --change-batch file://failover.json
```

D. AWS Services for DR & HA

Service	Role in HA/DR
S3	Cross-region replication, versioning for DR
EFS/EBS	EBS snapshots, EFS replication
RDS	Multi-AZ, cross-region read replicas
Aurora Global DB	Near-zero RPO multi-region failover
Route 53	Health checks, DNS-based failover
Elastic Load Balancer	Spreads traffic across AZs
EC2 Auto Scaling	Replaces failed instances automatically
Elastic Disaster Recovery (DRS)	Real-time replication of workloads
Backup	Centralized backup for EBS, RDS, DynamoDB

E. Elastic Disaster Recovery (AWS DRS)

- Near real-time replication from on-prem or AWS
- Failover from source to target in a few minutes
- Automatic orchestration of launch sequences

```
aws drs create-replication-configuration-template \
  --target-region us-west-2 \
  --server-type EC2 \
```

```
--staging-area-tags '{"Purpose":"dr"}'
```



F. Backup Strategies

Use **AWS Backup** for:

- Centralized, automated backup
- Retention policies
- Cross-region copy

```
resource "aws_backup_plan" "rds_plan" {
  name = "daily-rds-backup"
  rule {
    rule_name      = "daily"
    target_vault_name = "default"
    schedule       = "cron(0 5 * * ? *)"
    lifecycle {
      delete_after = 30
    }
  }
}
```



G. Monitoring DR Health

- **Route 53 health checks** for endpoint availability
- **CloudWatch alarms** for latency, error spikes
- **AWS Config rules** to verify DR configurations

```
aws route53 create-health-check \
  --caller-reference "failoverCheck" \
  --health-check-config file://config.json
```



Best Practices Summary

Strategy Area	Best Practice
AZ Distribution	Deploy across at least 2 AZs for all critical resources
DR Planning	Choose pattern based on RTO/RPO requirements
Backup Management	Use AWS Backup with lifecycle rules & cross-region copy
DNS Failover	Use Route 53 health checks and weighted failover
Data Resilience	Use EBS/EFS snapshots, RDS replicas, and DynamoDB streams
Audit Readiness	Enable AWS Config to monitor HA/DR compliance

16. CI/CD & DevOps Best Practices in AWS System Design

A resilient system design isn't just about uptime or backups — it's about **automated, consistent, and secure delivery** of software. Continuous Integration and Continuous Delivery/Deployment (CI/CD) are critical to achieve **agility without compromising reliability**.

In this section, we'll walk through:

- AWS-native DevOps tools
- CI/CD architecture patterns
- Deployment strategies (Blue/Green, Canary, Rolling)
- Terraform, CLI, SDK examples for implementation

A. What Is CI/CD?

Phase	Description
CI (Integration)	Automating the build, test, and packaging of your application every time code is committed
CD (Delivery)	Automating deployment to staging or production
CD (Deployment)	Automatically or manually deploying to production once delivery is successful

B. DevOps Tools in AWS

Tool	Purpose
CodeCommit	Managed Git repositories
CodeBuild	Build & test your code
CodeDeploy	Automate app deployment (EC2, Lambda, ECS)

CodePipeline	Orchestrates CI/CD flow
CodeArtifact	Private package repository
CloudWatch Logs & Alarms	Observability and failure alerting
IAM + Secrets Manager	Secure role-based permissions & secrets

C. CI/CD Architecture in AWS

Example Components:

```
Dev -> CodeCommit (Git) -> CodePipeline ->
    -> CodeBuild -> Unit Tests
    -> Deploy to Staging (CodeDeploy/Lambda/ECS)
    -> Manual Approval
    -> Production Deployment
```

Example: Terraform Pipeline (CodeCommit to Lambda)

```
resource "aws_codecommit_repository" "repo" {
  repository_name = "lambda-app"
  description     = "My serverless app"
}

resource "aws_codepipeline" "pipeline" {
  name = "lambda-ci-cd"

  role_arn = aws_iam_role.codepipeline_role.arn

  artifact_store {
    location = aws_s3_bucket.artifacts.bucket
    type     = "S3"
  }

  stage {
    name = "Source"
    action {
      name = "Source"
    }
  }
}
```

```

        category          = "Source"
        owner              = "AWS"
        provider           = "CodeCommit"
        version            = "1"
        output_artifacts   = ["source_output"]
        configuration = {
            RepositoryName = aws_codecommit_repository.repo.name
            BranchName      = "main"
        }
    }
}

stage {
    name = "Build"
    action {
        name          = "Build"
        category       = "Build"
        owner          = "AWS"
        provider        = "CodeBuild"
        input_artifacts = ["source_output"]
        output_artifacts = ["build_output"]
        configuration = {
            ProjectName = aws_codebuild_project.project.name
        }
    }
}

stage {
    name = "Deploy"
    action {
        name          = "Deploy"
        category       = "Deploy"
        owner          = "AWS"
        provider        = "Lambda"
        input_artifacts = ["build_output"]
        configuration = {
            FunctionName = aws_lambda_function.my_lambda.function_name
        }
    }
}

```

```
}  
}  
}
```

D. Testing Strategies

Test Type	Purpose
Unit Tests	Run during CodeBuild
Integration Tests	Use mock services (e.g., DynamoDB Local)
Load Tests	Run with tools like Artillery, Gatling, or JMeter
Security Tests	SAST/DAST tools like SonarQube, AWS Inspector

E. Deployment Strategies

1. Blue/Green Deployment

Two separate environments (blue=current, green=new). Traffic is switched after tests pass.

```
# CodeDeploy AppSpec example for Lambda  
version: 0.0  
Resources:  
  - myLambdaFunction:  
    Type: AWS::Lambda::Function  
    Properties:  
      Name: "my-lambda-fn"  
      Alias: "live"  
      CurrentVersion: "1"  
      TargetVersion: "2"
```

Can be triggered in CodeDeploy through CodePipeline.

2. Canary Deployment

Gradually shift traffic from old to new version.

Lambda Example:

```
aws lambda update-alias \  
  --function-name my-lambda-fn \  
  --name live \  
  --routing-config '{"AdditionalVersionWeights": {"2": 0.1}}'
```

- Initially route 10% to v2, then 50%, then 100%
- Roll back on failure using CloudWatch Alarms + CodeDeploy

3. Rolling Deployment

Gradually replaces old instances (used in EC2, ECS, ASG)

```
resource "aws_autoscaling_group" "asg" {  
  ...  
  min_size           = 2  
  max_size           = 4  
  desired_capacity   = 2  
  rolling_update_policy {  
    max_batch_size      = 1  
    min_instances_in_service = 1  
    pause_time          = "PT5M"  
  }  
}
```

F. Security in CI/CD

- Use **IAM roles** for each pipeline stage (principle of least privilege)
- Store secrets in **Secrets Manager** or **Parameter Store**
- Enforce **code scanning** in the build stage
- **Enable artifact integrity checks** using SHA checksums

G. DevOps Monitoring & Rollback

Tool	Usage
CloudWatch	Alert on failed builds, deployment errors
CodePipeline Logs	Analyze failed stages
SNS + Lambda	Notify Slack/Teams on failure
CodeDeploy Rollback	Automatic rollback if health check fails

Best Practices Summary

Area	Practice
Code Source	Use CodeCommit or GitHub
Build Automation	Use CodeBuild with isolated environments
Deployment	Prefer Blue/Green or Canary over in-place
Security	Use IAM roles, least privilege, no hardcoded creds
Testing	Automate unit, integration, and security tests
Observability	Hook CloudWatch alarms to rollback triggers
Audit Trail	Enable AWS CloudTrail + CodePipeline logs retention

17. Performance Optimization in AWS

System Design

Performance optimization isn't about throwing more compute at a problem. It's about **intelligently tuning** architecture components — networking, storage, compute, databases, and caching — to handle varying workloads **efficiently**.

A. Why Performance Optimization Matters

- Ensures **low latency**, **high throughput**, and **smooth scaling**.
- Reduces **costs** by preventing overprovisioning.
- Supports **business SLAs** and **user expectations**.
- Identifies **bottlenecks** before they cause downtime.

B. Key Optimization Areas

Area	Goal	Tools / Services
Compute	Right-size and scale	EC2, ASG, Lambda
Storage	Low-latency access	EBS, EFS, S3
Database	Fast queries, scalability	RDS, DynamoDB, Aurora
Networking	Low latency, efficient routing	VPC, CloudFront, Global Accelerator
Caching	Reduce backend load	ElastiCache, API Gateway caching
Application Layer	Faster code execution	Lambda tuning, container resource limits

C. Compute Optimization

◆ EC2 Right-Sizing

Choose instance families based on your workload (e.g., `c6g` for compute, `r6i` for memory).

Use AWS Compute Optimizer:

```
aws compute-optimizer get-ec2-instance-recommendations
```

Auto Scaling Example:

```
resource "aws_autoscaling_group" "asg" {
  min_size      = 2
  max_size      = 10
  desired_capacity = 2

  launch_template {
    id      = aws_launch_template.lt.id
    version = "$Latest"
  }

  tag {
    key          = "Name"
    value        = "app-server"
    propagate_at_launch = true
  }
}
```

◆ Lambda Performance Tuning

- Set memory higher → automatically improves CPU.
- Measure cold start and execution duration using CloudWatch.

```
aws lambda update-function-configuration \
  --function-name my-fn \
  --memory-size 1024 \
  --timeout 10
```

Use **Provisioned Concurrency** to avoid cold starts:

```
aws lambda put-provisioned-concurrency-config \
```

```
--function-name my-fn \  
--qualifier "prod" \  
--provisioned-concurrent-executions 10
```

D. Storage Optimization

Use Case	Recommendation
High IOPS	Use io2 volumes with provisioned IOPS
Shared file systems	Use EFS with throughput mode tuning
Archived data	Use S3 Glacier or Deep Archive
Frequent reads	Use S3 Transfer Acceleration or CloudFront

EBS Optimized Volume Example:

```
resource "aws_ebs_volume" "optimized" {  
  availability_zone = "us-east-1a"  
  size              = 100  
  type              = "io2"  
  iops              = 10000  
  throughput        = 500  
  tags = {  
    Name = "high-performance-vol"  
  }  
}
```

E. Database Optimization

◆ Amazon RDS

- Use **Read Replicas** for read-heavy workloads.
- Enable **Query Caching** at the DB engine level.
- Monitor slow queries with **Performance Insights**.

```
aws rds create-db-instance-read-replica \  

```

```
--db-instance-identifier my-db-ro \  
--source-db-instance-identifier my-db
```

◆ Amazon DynamoDB

- Use **Provisioned with Auto Scaling** or **On-Demand**.
- Use **DAX (DynamoDB Accelerator)** for in-memory cache.
- Optimize partition keys to avoid hot partitions.

```
aws dynamodb update-table \  
--table-name MyTable \  
--provisioned-throughput ReadCapacityUnits=10,WriteCapacityUnits=10
```

F. Network Optimization

Optimization Goal	Recommended Approach
Reduce latency globally	Use Amazon CloudFront , Global Accelerator
Minimize VPC traffic costs	Prefer interface endpoints (VPCe)
Accelerate uploads	Enable S3 Transfer Acceleration
Secure & fast internal traffic	Use PrivateLink , Transit Gateway

G. Caching Optimization

Layer	Tool
App-level	Redis or Memcached with ElastiCache
API-level	API Gateway with response caching
DB-level	DAX for DynamoDB or Query Caching in RDS
Web assets	CloudFront with cache control headers

ElastiCache Redis Example:

```
resource "aws_elasticache_cluster" "redis" {  
  cluster_id      = "redis-cluster"
```

```
engine            = "redis"
node_type         = "cache.t3.micro"
num_cache_nodes   = 1
parameter_group_name = "default.redis6.x"
}
```

H. Observability for Optimization

Use **CloudWatch Dashboards, Logs, and Metrics** to track:

- CPU/Memory utilization
- Duration and invocations (Lambda)
- Read/Write IOPS (EBS, RDS)
- API Gateway latency
- Cache hit ratio (ElastiCache)

Sample CLI:

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda \
  --metric-name Duration \
  --dimensions Name=FunctionName,Value=my-function \
  --start-time 2025-06-14T00:00:00Z \
  --end-time 2025-06-14T23:59:59Z \
  --period 3600 \
  --statistics Average
```

✓ Best Practices Summary

Optimization Area	Best Practices
Compute	Right-size EC2, auto-scale Lambda, tune concurrency
Storage	Use EBS types wisely, tier S3 data
DB	Read replicas, query tuning, DynamoDB partitioning
Networking	Use CloudFront, Global Accelerator, VPC endpoints
Caching	Cache aggressively with ElastiCache/DAX
Monitoring	Set CloudWatch alerts, track KPIs, enable profiling



18. Observability & Logging Architecture in AWS

"You can't fix what you can't see."

In modern cloud-native architectures, observability isn't a luxury — it's essential. It enables **real-time monitoring, root cause analysis, alerting, and proactive response** to performance, availability, and security issues.

This section explores:

- Core observability components in AWS
- Centralized logging best practices
- Real-time monitoring and alerting
- Metrics, traces, logs (the three pillars)
- IaC examples using Terraform, CLI, and AWS SDK



A. Three Pillars of Observability

Pillar	Purpose	AWS Services
Logs	Records of events over time	CloudWatch Logs, OpenSearch, S3
Metrics	Numerical data to detect anomalies	CloudWatch Metrics, Custom Metrics
Traces	Follow a single transaction end-to-end	AWS X-Ray, OpenTelemetry, AppConfig



B. Metrics Collection



Native Metrics with CloudWatch

Every AWS service emits metrics to **Amazon CloudWatch** by default.

```
aws cloudwatch list-metrics --namespace "AWS/EC2"
```

You can create **custom metrics** (e.g., memory usage in EC2):

```
aws cloudwatch put-metric-data \  
  --namespace "Custom/EC2" \  
  --metric-name MemoryUtilization \  
  --dimensions InstanceId=i-12345678 \  
  --value 78.2
```

Infrastructure-as-Code (Terraform):

```
resource "aws_cloudwatch_metric_alarm" "high_cpu" {  
  alarm_name      = "HighCPUALarm"  
  comparison_operator = "GreaterThanThreshold"  
  evaluation_periods = "2"  
  metric_name     = "CPUUtilization"  
  namespace      = "AWS/EC2"  
  period         = "60"  
  statistic      = "Average"  
  threshold      = "80"  
  alarm_description = "This metric monitors high CPU usage"  
  dimensions = {  
    InstanceId = "i-12345678"  
  }  
}
```

C. Logging Strategies

◆ CloudWatch Logs (Default)

- Stores logs from Lambda, ECS, EC2, API Gateway, etc.
- Supports retention policies, filters, and insights

```
aws logs create-log-group --log-group-name "app-logs"  
aws logs create-log-stream --log-group-name "app-logs" --log-stream-name  
"api-stream"  
aws logs put-retention-policy \  

```

```
--log-group-name "app-logs" \  
--retention-in-days 14
```

◆ S3-Based Logging (Long-Term)

- Store audit, access, and application logs for years
- Use S3 lifecycle policies for cost management

```
aws s3api put-bucket-logging \  
--bucket mybucket \  
--bucket-logging-status '{  
  "LoggingEnabled": {  
    "TargetBucket": "my-logging-bucket",  
    "TargetPrefix": "logs/"  
  }  
'
```

◆ OpenSearch (Log Analytics)

Use **Amazon OpenSearch Service** to ingest logs and run full-text search or visualize dashboards (alternative to ELK stack).

D. Tracing & Distributed Monitoring

◆ AWS X-Ray

Provides detailed insight into request flows across microservices (Lambda, API Gateway, ECS, EC2).

```
# SDK Example (Python)  
from aws_xray_sdk.core import xray_recorder, patch_all  
patch_all()  
  
@xray_recorder.capture('my_handler')  
def lambda_handler(event, context):
```

```
# Automatically traced
return {"status": "ok"}
```

◆ OpenTelemetry

- Open standard supported in AWS Distro for OpenTelemetry (ADOT)
- Useful for ECS, EKS, container-based microservices

Terraform setup example:

```
resource "aws_eks_addon" "otel" {
  cluster_name = "my-eks-cluster"
  addon_name    = "adot-collector"
  addon_version = "v0.85.0-eksbuild.1"
}
```

E. Dashboards and Visualization

◆ CloudWatch Dashboards

```
aws cloudwatch put-dashboard \
  --dashboard-name "MyAppDashboard" \
  --dashboard-body '{
    "widgets": [{
      "type": "metric",
      "x": 0,
      "y": 0,
      "width": 6,
      "height": 6,
      "properties": {
        "metrics": [
          ["AWS/EC2", "CPUUtilization", "InstanceId", "i-12345678"]
        ],
        "period": 300,
        "stat": "Average",
        "region": "us-east-1",
        "title": "EC2 CPU Usage"
      }
    }
  ]
}
```

```
}  
  }]  
}'
```

◆ Third-Party Tools Integration

Tool	Integration Mode
Datadog	CloudWatch metrics/logs + Lambda layers
New Relic	OpenTelemetry exporters
Prometheus	EKS Fargate and EC2 with exporters
Grafana	Supports CloudWatch + OpenSearch

⚠ F. Alerting and Incident Response

◆ CloudWatch Alarms + SNS

Send alert emails or trigger Lambda automation on metric thresholds.

```
aws sns create-topic --name "ops-alerts"  
aws sns subscribe --topic-arn arn:aws:sns:... --protocol email --  
notification-endpoint you@example.com  
resource "aws_cloudwatch_metric_alarm" "lambda_error" {  
  alarm_name      = "LambdaErrorAlarm"  
  comparison_operator = "GreaterThanThreshold"  
  evaluation_periods = "1"  
  threshold        = 1  
  metric_name       = "Errors"  
  namespace         = "AWS/Lambda"  
  statistic         = "Sum"  
  period           = 60  
  alarm_actions     = [aws_sns_topic.alerts.arn]  
}
```

◆ Automation with Lambda or SSM

```
# Example: Restart ECS Task on alarm
aws events put-rule --name "HighErrorRate"
aws lambda add-permission ...
aws events put-targets ...
```



G. Observability Architecture Example

```
[Lambda / ECS] --> CloudWatch Logs --> Log Insights + S3 Archive
               --> CloudWatch Metrics --> Alarms + Dashboards
               --> X-Ray Traces
               --> SNS Alerts --> Email / Slack / Lambda Automation
               --> Optional: Forward to OpenSearch or Grafana
```



Best Practices Summary

Principle	Best Practice
Centralized Logging	Use CloudWatch Logs + S3 + OpenSearch
Retention Management	Apply log retention and lifecycle policies
Traceability	Enable X-Ray or OpenTelemetry on all services
Dashboards	Use CloudWatch or Grafana for ops visualization
Alerting	Use CloudWatch + SNS + Lambda automation
Cost Efficiency	Archive old logs in S3 Deep Archive
Security & Access	Use IAM policies, encrypt logs with KMS, audit log access

19. Designing for Global Availability and Multi-Region Deployments

Why Multi-Region Architecture?

Multi-Region architecture helps achieve:

- **Disaster recovery (DR)** and **business continuity**
- **Low-latency user experience** worldwide
- Compliance with **data sovereignty** laws
- Enhanced **fault isolation**

However, it also brings **complexity**, **cost**, and **replication consistency** challenges — which we'll tackle here.

A. Core AWS Services Used for Global Design

Use Case	AWS Service
Global DNS & traffic routing	Amazon Route 53
Multi-region content delivery	Amazon CloudFront
Application acceleration	AWS Global Accelerator
Replication of data	S3 Cross-Region Replication, Aurora Global Databases, DynamoDB Global Tables
Stateful failover	Route 53 Health Checks + Failover
Disaster recovery automation	AWS Backup, CloudFormation StackSets, AWS DRS



B. DNS Routing & Global Traffic Control

1. Amazon Route 53

Use **geolocation**, **latency-based**, and **failover routing** policies to route global traffic intelligently.

```
# Create health check
aws route53 create-health-check --caller-reference "hc1" \
  --health-check-config '{
    "IPAddress": "34.220.12.34",
    "Port": 80,
    "Type": "HTTP",
    "ResourcePath": "/health",
    "RequestInterval": 30,
    "FailureThreshold": 3
  }'
```

2. Latency-Based Routing

Let's say you deploy your app in `us-east-1` and `eu-west-1` — Route 53 automatically directs users to the closest endpoint.

Terraform Example:

```
resource "aws_route53_record" "latency_routing" {
  zone_id = aws_route53_zone.primary.zone_id
  name     = "app.example.com"
  type     = "A"
  set_identifier = "us-east"
  latency_routing_policy {
    region = "us-east-1"
  }
  alias {
```

```
name                = aws_lb.app_lb.dns_name
zone_id             = aws_lb.app_lb.zone_id
evaluate_target_health = true
}
}
```

C. Content Delivery & Network Acceleration

1. Amazon CloudFront

Use for globally distributed content caching (e.g., websites, images, APIs).

```
aws cloudfront create-distribution --origin-domain-name
myapp.s3.amazonaws.com
```

2. AWS Global Accelerator

Optimizes TCP/UDP path globally — ideal for gaming, APIs, low-latency apps.

Benefits:

- Static IPs
- Intelligent global routing
- Health-check-based failover

D. Data Replication Strategies

1. S3 Cross-Region Replication (CRR)

```
aws s3api put-bucket-replication \
--bucket my-source-bucket \
--replication-configuration '{
  "Role": "arn:aws:iam::123456789012:role/s3-replication-role",
  "Rules": [{
    "Status": "Enabled",
```

```

    "Prefix": "",
    "Destination": {
        "Bucket": "arn:aws:s3:::my-destination-bucket"
    }
  }]
}'

```

2. Aurora Global Database

Replicates to secondary Regions with <1s latency. Ideal for RPO $\approx$ 0 systems.

```

aws rds create-global-cluster \
  --global-cluster-identifier my-global-db \
  --source-db-cluster-identifier arn:aws:rds:us-east-
1:123456789012:cluster:aurora-cluster-1

```

3. DynamoDB Global Tables

Multi-active write and read across regions.

```

aws dynamodb create-global-table \
  --global-table-name Products \
  --replication-group RegionName=us-east-1 RegionName=ap-south-1

```

E. Disaster Recovery (DR) Strategies

DR Strategy	RTO	RPO	Cost	Description
Backup & Restore	Hours	Hours	Low	Periodic backups (e.g. to S3, RDS)
Pilot Light	Minutes	Minutes	Medium	Minimal infra pre-provisioned
Warm Standby	Minutes	Seconds	Higher	Scaled-down active replica ready
Active-Active	Seconds	Seconds	Highest	Fully operational across regions

Use AWS Elastic Disaster Recovery (DRS)

Continuously replicates EC2 workloads between regions.

```
aws drs create-source-server --source-properties ...
```

F. Infrastructure Deployment & Sync

1. CloudFormation StackSets

Deploy the same stack (e.g., VPC, ECS, IAM) across multiple regions/accounts.

```
aws cloudformation create-stack-set \  
  --stack-set-name MultiRegionInfra \  
  --template-body file://template.yaml \  
  --permission-model SELF_MANAGED
```

2. CI/CD Deployment Strategy

Use GitHub Actions, CodePipeline, or Jenkins to deploy region-specific stacks.

```
# GitHub Action Snippet  
- name: Deploy to us-east-1  
  run: |  
    aws cloudformation deploy \  
      --template-file infra.yaml \  
      --stack-name myapp-us \  
      --region us-east-1  
  
- name: Deploy to eu-west-1  
  run: |  
    aws cloudformation deploy \  
      --template-file infra.yaml \  
      --stack-name myapp-eu \  
      --region eu-west-1
```

G. Synchronizing State & Handling Failover

- Use **Route 53 failover routing + health checks**
- Maintain consistent **S3, DynamoDB, and Secrets** across regions

- Use **event-driven sync** with Amazon EventBridge or Step Functions

H. Common Design Patterns

Active-Active Web App with Global Load Balancing

User → Route 53 → Global Accelerator → CloudFront → App in closest Region → Global DB or S3 CRR

Global Microservices with Regional Failover

User → Route 53 Latency-Based Routing
→ us-east-1 (primary ECS/Lambda)
→ eu-central-1 (standby)

Best Practices Summary

Area	Recommendation
DNS & Routing	Route 53 latency + failover policies
Performance	CloudFront + Global Accelerator
Data Replication	Use Global Tables / Aurora Global DB / S3 CRR
Resiliency	Implement health checks, alarms, failover
Infrastructure as Code	Use StackSets, region-based CI/CD
Observability	Aggregate logs + metrics across regions
Cost Control	Use warm standby instead of full active-active unless justified

20. Real-World System Design Patterns & Architectures in AWS

"Architecture is not just about tools — it's about the right composition."

Now that we've understood the individual AWS components and design principles, it's time to piece them together into **battle-tested system patterns**. These are drawn from real-world deployments across industries like fintech, e-commerce, SaaS, healthcare, and media streaming.

This section focuses on **common architectural blueprints**, why they're used, and how to implement them using AWS CLI, SDKs, or IaC (Terraform/CloudFormation).

A. Event-Driven Architecture (EDA)

◆ When to Use

- Microservices that need to loosely couple
- Async processing (e.g., image processing, order fulfillment)
- Scalability and reliability

◆ AWS Services Involved

- Amazon SNS (pub/sub)
- Amazon SQS (decoupling queues)
- AWS Lambda / Step Functions (processing logic)
- EventBridge (event bus)

Architecture Diagram

Event Source → SNS → SQS → Lambda → Database/Service

↓

Other SQS (for retries or parallel consumers)

Terraform Snippet

```
resource "aws_sns_topic" "events" {
  name = "order-events"
}

resource "aws_sqs_queue" "order_queue" {
  name = "order-queue"
}

resource "aws_sns_topic_subscription" "sub" {
  topic_arn = aws_sns_topic.events.arn
  protocol  = "sqs"
  endpoint  = aws_sqs_queue.order_queue.arn
}
```

B. CQRS + Event Sourcing

◆ When to Use

- Applications requiring audit logs, temporal state
- Systems with distinct read/write models (e.g., fintech, inventory)

Design Pattern

Command API → Lambda → Event Store (S3/DynamoDB)

↓

Read Model updated asynchronously
via Lambda + DynamoDB

AWS Implementation

- **Event store:** S3 with object versioning OR DynamoDB streams

- **Read model:** DynamoDB table with GSI for fast querying
- **Trigger:** Lambda from EventBridge or DynamoDB Streams

C. Serverless Microservices

◆ When to Use

- Fast startup, minimal operations
- Spiky workloads
- Rapid experimentation

Common Stack

Layer	AWS Service
API Layer	Amazon API Gateway
Logic	AWS Lambda
Data	DynamoDB / Aurora Serverless
Events	EventBridge / SNS
Security	Cognito + IAM + WAF
Observability	CloudWatch + X-Ray

Example CLI Setup

```
aws apigateway create-rest-api --name "MicroAPI"
aws lambda create-function --function-name userHandler ...
aws apigateway put-integration ...
```

D. E-Commerce Web Architecture

◆ Requirements

- Product catalog, cart, checkout, order tracking

- Scale on traffic bursts
- Payment integration

AWS Blueprint

```
[CloudFront]
  ↓
[API Gateway + Cognito]
  ↓
[Lambdas for each microservice]
  ↓
[DynamoDB or Aurora Serverless]
  ↓
[SNS/SQS for async processes like invoice/email]
```

Terraform Snippet for S3 Static Website + CloudFront

```
resource "aws_s3_bucket" "site" {
  bucket = "shop-app"
  website {
    index_document = "index.html"
  }
}

resource "aws_cloudfront_distribution" "cdn" {
  origin {
    domain_name = aws_s3_bucket.site.bucket_regional_domain_name
    origin_id   = "s3-site"
  }

  enabled          = true
  default_root_object = "index.html"
}
```

E. Real-Time Streaming with Analytics

◆ Use Cases

- Media content delivery (e.g., OTT apps)
- Real-time analytics on user behavior
- Social engagement tracking

AWS Architecture

```
[Video Source / App Events]
↓
[Kinesis Data Streams / Firehose]
↓
[Lambda / Analytics Pipeline]
↓
[S3 / Redshift / QuickSight Dashboard]
```

AWS CLI Kinesis Example

```
aws kinesis create-stream --stream-name analyticsStream --shard-count 1
```

F. Zero Trust Security Architecture

◆ AWS Implementation

Component	AWS Services
Identity & Access	AWS IAM, AWS SSO, Cognito
Resource Protection	VPC, Security Groups, WAF
Data Protection	KMS, IAM Roles, Secrets Manager
Threat Detection	GuardDuty, Macie, Inspector
Monitoring	CloudTrail, Config, Detective

G. Hybrid Cloud / On-Prem + AWS

◆ Use Cases

- Gradual cloud adoption
- Regulatory constraints
- Cloud bursting or DR

AWS Tools

- **Direct Connect / VPN Gateway**
- **AWS Storage Gateway**
- **EKS Anywhere / Outposts**
- **Transit Gateway**

H. Service Mesh with AWS App Mesh

◆ When to Use

- Need for fine-grained traffic control
- Canary deployments
- Secure communication between services

Key Features

- Envoy sidecars
- Integration with EKS and ECS
- Works with CloudMap for service discovery

```
aws appmesh create-mesh --mesh-name "payments-mesh"
```

I. Multi-Tenant SaaS Platform

◆ Use Cases

- Host multiple tenants in isolation
- Pay-as-you-go billing
- Usage metering

AWS Design

- Cognito + IAM for tenant identity
- Separate DynamoDB partition or schema
- Shared Lambda or container services with tenant routing
- Metering with CloudWatch + Billing APIs

Summary: Choose the Right Pattern Based on Your Needs

Pattern	Best For
Event-Driven Architecture	Async decoupled systems
Serverless Microservices	Fast scaling, low ops overhead
CQRS + Event Sourcing	Complex workflows with audit needs
Real-Time Streaming + Analytics	Time-sensitive insight
Multi-Tenant SaaS	Scalable, isolated customer stacks
Hybrid Architecture	On-prem + AWS coexistence
Zero Trust Security	Enforced security & compliance



21. Case Studies & Real-World

Architecture Breakdowns

“Real-world success lies not just in choosing the right tools but in orchestrating them effectively under constraints.”

This section will detail **actual AWS-based architectures** deployed by organizations — from startups to enterprises — with challenges, architectural decisions, and AWS service stacks. These examples will help you think like a system architect facing real production constraints: scale, cost, latency, security, compliance, disaster recovery, and time-to-market.



A. E-Commerce Platform: Scaling for Black Friday



Client: Mid-size Retailer



Goal: Handle massive seasonal traffic spikes (20x) during flash sales



Requirements:

- High availability (HA)
- Seamless scaling under load
- Real-time stock updates
- Customer segmentation for promotions



Architecture:

Layer	AWS Services Used
Frontend CDN	Amazon CloudFront
Web Tier	Amazon ECS on Fargate (microservices)

API Gateway	Amazon API Gateway + AWS WAF
Backend	Amazon Aurora Serverless (MySQL)
Stock Sync	DynamoDB Streams + Lambda
Promo Engine	SQS + Lambda + Aurora
Observability	Amazon CloudWatch + X-Ray

Flow Diagram:

```

User → CloudFront → API Gateway → ECS (Service: cart, orders, users)
                                ↓
                        Aurora / DynamoDB
                                ↓
                    SQS → Lambda → Promotions Engine
  
```

Challenge Solved:

- Avoided DB overload by moving real-time stock checks to DynamoDB
- Promo engine processed thousands of records/min using SQS+Lambda
- Elastic scalability with Fargate + ALB ensured no downtime

B. Video Streaming Startup: Global Media Delivery

Client: SaaS OTT Platform

Goal: Deliver low-latency video globally with analytics insights

Requirements:

- Global content delivery
- Edge caching
- Viewer engagement analytics
- Real-time transcoding and delivery

Architecture:

Component	AWS Services
Video Upload	Amazon S3 + S3 Multipart Upload
Transcoding	AWS Elemental MediaConvert
Delivery	Amazon CloudFront + Signed URLs
Analytics	Amazon Kinesis + Redshift + QuickSight
Notifications	EventBridge + Lambda + SNS

Streaming Flow:

```
Creator Upload → S3 → Event → MediaConvert → S3 (Optimized)
                    ↓
                CloudFront (Edge Delivery) → End User
                    ↓
            Viewer data → Kinesis → Redshift → Dashboard
```

Challenge Solved:

- Delivered <3s latency across 5 continents using CloudFront POPs
- Tracked viewer engagement in near real-time
- Used signed URLs to protect premium content access

C. FinTech App: Secure, Compliant Multi-Tenant SaaS

Client: B2B SaaS FinTech Provider

 **Goal:** Offer financial dashboards to banks with complete tenant isolation and regulatory compliance

Requirements:

- Data encryption at rest and in transit
- Isolation between tenant data
- Audit logs and fine-grained access control
- Multi-region backup

Architecture:

Requirement	AWS Solution
Identity Federation	AWS Cognito + IAM + SAML
API Gateway	Amazon API Gateway (per-tenant custom domains)
Application Logic	AWS Lambda + Step Functions
Storage	DynamoDB (partitioned by tenant) + KMS
Audit Logs	CloudTrail + Athena
Backup & Compliance	AWS Backup + S3 + Glacier

Security Practices:

- Per-tenant KMS CMKs
- Lambda runs with scoped IAM roles
- VPC endpoint access only (no public access)

Challenge Solved:

- Onboarded new tenants in under 5 minutes with automation
- Maintained GDPR/PCI-DSS compliance using native AWS tools
- Reduced cost by avoiding multiple VPCs through logical isolation

D. Healthcare Analytics: HIPAA-Compliant Data Lake

Client: National Hospital Chain

 **Goal: Collect, store, and analyze sensitive patient data securely across 80+ clinics**

 Requirements:

- HIPAA compliance
- PII/PHI encryption and classification
- Unified data lake with granular access control
- Predictive analytics

 Architecture:

Component	AWS Service
Data Ingestion	AWS DMS (from on-prem), AWS Glue
Data Lake	Amazon S3 + Lake Formation
Security	Macie + IAM + KMS + Secrets Manager
Processing	Amazon Athena + Amazon EMR (Spark)
Dashboarding	Amazon QuickSight

 Security Controls:

- PII detection via **Amazon Macie**
- Encrypted data in S3 (KMS CMKs)
- Lake Formation access policies based on roles
- Cross-account access audits via AWS Config

 Challenge Solved:

- Enabled self-service analytics across departments without exposing PII
- Reduced compliance audit response time from 2 weeks to 2 hours

 E. Multi-Region SaaS Platform: Always-On Uptime

 **Client: Global SaaS CRM Provider**

 **Goal: Ensure zero downtime with multi-region failover & fast user experience**

 **Requirements:**

- Active-Active architecture
- Global user routing
- Data consistency
- Sub-second failover

 **Architecture:**

Function	AWS Services
Traffic Routing	Route 53 + AWS Global Accelerator
App Hosting	ECS on Fargate (us-east-1, eu-west-1)
DB Layer	Aurora Global Database + S3 CRR
Sync + State	EventBridge + DynamoDB Global Tables
CI/CD	CodePipeline with region-specific workflows

 **Global Setup Flow:**

User → Route 53 → Nearest Region (via Global Accelerator)

↓

ECS App + Aurora Global DB + DynamoDB

↓

Failover if unhealthy via Route 53 Health Checks

 **Challenge Solved:**

- Achieved SLA of 99.999% globally
- Reduced latency by 40% for EU/India users

Written by: MANISH KUMAR

Page | 142

- Aurora Global DB with <1s replication lag kept consistency in sync



Lessons Learned Across Case Studies

Challenge	Best AWS Solution
Traffic Spikes	Auto Scaling + Serverless (Fargate/Lambda)
Global Delivery	CloudFront + Global Accelerator
Data Protection	KMS + IAM + Macie + VPC Endpoints
Event-Driven Scalability	SQS/SNS + Lambda
Multi-Region High Availability	Route 53 + Aurora Global + Global Tables
Observability	CloudWatch + X-Ray + CloudTrail + Config



22. AWS System Design Architect – One-Liner Interview Q&A

Q: What is the difference between EC2 and ECS?

A: EC2 is a virtual server; ECS is a container orchestration service that can run on EC2 or Fargate.

Q: Why use S3 over EBS or EFS?

A: S3 is object storage, EBS is block storage for EC2, and EFS is shared POSIX-compliant file storage.

Q: What are the key pillars of AWS Well-Architected Framework?

A: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

Q: How do you decide between vertical and horizontal scaling?

A: Vertical scaling adds power to existing machines; horizontal scaling adds more machines for resilience and scalability.

Q: What is the purpose of a NAT Gateway?

A: Allows private subnets to access the internet without receiving inbound traffic.

Q: Difference between Security Groups and NACLs?

A: SGs are stateful and instance-level; NACLs are stateless and subnet-level.

Q: When to use S3 Intelligent-Tiering?

A: For infrequently accessed data with unknown access patterns.

Q: How do you ensure secure access to S3 buckets?

A: Use IAM policies, bucket policies, and block public access settings.

Q: When to use AWS Lambda over Fargate?

A: For event-driven, short-lived tasks (<15 mins) with no server management.

Q: What's a benefit of using EC2 Auto Scaling?

A: Automatically adjusts capacity to maintain performance and cost.

Q: What is the principle of least privilege in IAM?

A: Grant the minimum permissions needed to perform a task.

Q: What are IAM Roles used for?

A: To grant temporary access to AWS resources without sharing credentials.

Q: How do you detect threats in AWS?

A: Use GuardDuty, Macie, AWS Config, and CloudTrail.

Q: What is AWS KMS used for?

A: Managing encryption keys for data protection across AWS services.

Q: What's the difference between HA and fault tolerance?

A: HA reduces downtime; fault tolerance ensures continuous operation during failure.

Q: How do you build HA across regions?

A: Use Route 53, Global Accelerator, and services like Aurora Global Database.

Q: What is a stateless application and why is it important for scaling?

A: Doesn't rely on local state; allows easy scaling across instances.

Q: How can DynamoDB scale automatically?

A: Using On-Demand mode or auto-scaling with provisioned capacity.

Q: How do you trace distributed apps in AWS?

A: Use AWS X-Ray for tracing and CloudWatch Logs for centralized logging.

Q: How is AWS CloudTrail different from CloudWatch?

A: CloudTrail logs API activity; CloudWatch monitors metrics and logs.

Written by: MANISH KUMAR

Q: How do you reduce compute cost in AWS?

A: Use Spot Instances, Savings Plans, and auto-scaling.

Q: Which tool helps track and visualize AWS costs?

A: AWS Cost Explorer and AWS Budgets.

Q: What service helps orchestrate deployments to EC2, ECS, and Lambda?

A: AWS CodeDeploy.

Q: How do you implement blue/green deployments in ECS?

A: Use CodeDeploy with ECS and Application Load Balancer support.

Q: What tool validates your CloudFormation templates before deployment?

A: `cfn-lint` or CloudFormation's Change Sets.

Q: How do you test failover in a multi-AZ setup?

A: Simulate instance or AZ failure and verify automatic recovery.

Q: What tool can you use to visually plan AWS architectures?

A: AWS Architecture Icons on draw.io, Lucidchart, or Cloudcraft.

Q: What AWS service offers embedded architectural guidance?

A: AWS Well-Architected Tool.

Q: What are the typical layers of a multi-tier web app in AWS?

A: Presentation (CloudFront/S3), Application (ECS/Lambda), and Data (RDS/DynamoDB).

Q: Why separate the layers?

A: For scalability, security, and maintainability.

Q: What's the difference between Redshift and Athena?

A: Redshift is a data warehouse; Athena queries data directly from S3.

Q: How do you move structured data into a data lake?

A: Use AWS Glue ETL or DMS for data migration.

Written by: MANISH KUMAR

Q: When to use SQS vs SNS?

A: SQS for queues (point-to-point), SNS for pub/sub (fan-out messaging).

Q: What is Amazon EventBridge used for?

A: Event bus for integrating SaaS and AWS events in a decoupled architecture.

Q: How do you ensure global low latency?

A: Use CloudFront, Global Accelerator, and regional resources.

Q: How to replicate S3 data across regions?

A: Use S3 Cross-Region Replication (CRR).

Q: What are the four AWS DR strategies?

A: Backup & Restore, Pilot Light, Warm Standby, Multi-Site.

Q: How do you backup data on AWS?

A: Use AWS Backup, RDS snapshots, and S3 lifecycle policies.

Q: What's the benefit of Event-Driven Architecture?

A: Enables loose coupling, scalability, and async processing.

Q: What is the use case of a Serverless Microservices pattern?

A: Rapid scale, minimal ops, ideal for startups and spiky traffic.

Q: How can you build a HIPAA-compliant data lake?

A: Use S3 + Macie + KMS + Lake Formation + IAM policies.

Q: How do e-commerce sites scale for flash sales?

A: ECS/Fargate with Auto Scaling + SQS for queueing + Aurora Serverless.

Q: How do you decouple microservices in AWS?

A: Use Amazon SQS, SNS, EventBridge, or API Gateway for loose coupling.

Q: What's a common bottleneck in monolith migration to AWS?

A: Shared database layer and tightly coupled code logic.

Written by: MANISH KUMAR

Q: What tool helps visualize architectural dependencies?

A: AWS CloudWatch ServiceLens or AWS X-Ray.

Q: What is the cold start problem in AWS Lambda?

A: Delay during first invocation due to function container initialization.

Q: How do you reduce Lambda cold starts?

A: Use provisioned concurrency or keep functions warm via scheduled events.

Q: When should you choose Step Functions over Lambda chaining?

A: When orchestration, retry logic, and state tracking are required.

Q: How is ECS different from EKS?

A: ECS is AWS-managed; EKS provides Kubernetes compatibility.

Q: What is the advantage of using Fargate?

A: Serverless compute for containers — no server provisioning needed.

Q: How do you perform a rolling deployment in ECS?

A: Use ECS services with Application Load Balancer and deployment configurations.

Q: How do you ensure least privilege at scale in IAM?

A: Use IAM Access Analyzer and fine-grained policies scoped to services and conditions.

Q: How can VPC endpoints improve security?

A: They allow private connectivity to AWS services without using public internet.

Q: How can you prevent accidental S3 data exposure?

A: Enable “Block Public Access” and use service control policies in AWS Organizations.

Q: How does API Gateway integrate with Lambda?

A: API Gateway triggers Lambda functions on HTTP/S endpoints and manages request/response mapping.

Q: How can CloudWatch Logs be centralized from multiple accounts?

A: Use CloudWatch Logs cross-account subscription filters or Kinesis Firehose.

Q: How can you automate tagging of AWS resources?

A: Use AWS Organizations service control policies (SCPs) or automation with Lambda/Config Rules.

Q: How do you control runaway costs in development accounts?

A: Set up Budgets, enforce limits via SCPs, and auto-stop resources with Lambda.

Q: What are Compute Savings Plans?

A: Commitment-based pricing to reduce costs on EC2, Fargate, and Lambda.

Q: What is Spot Fleet used for?

A: To manage and provision spot instances across pools for cost-optimized workloads.

Q: How do you test the resiliency of a system on AWS?

A: Use chaos engineering tools like AWS Fault Injection Simulator.

Q: What is the difference between Multi-AZ and Multi-Region?

A: Multi-AZ ensures fault tolerance in one region; Multi-Region offers disaster recovery and global HA.

Q: What's the RTO and RPO for S3 cross-region replication?

A: Typically seconds to minutes; depends on bucket setup and region latency.

Q: What is the difference between Kinesis Data Streams and Firehose?

A: Streams is for real-time processing; Firehose for ingest and batch delivery to destinations.

Q: How do you ensure delivery ordering in Kinesis?

A: Use partition keys to ensure order within shards.

Q: How do you optimize Redshift query performance?

A: Use distribution styles, sort keys, and analyze compression.

Q: How do you ensure secure access to RDS?

A: Use IAM authentication, SSL/TLS, and restrict access via security groups.

Q: What is Aurora Global Database ideal for?

A: Low-latency, cross-region read replicas with fast disaster recovery.

Q: What's the role of AWS DMS?

A: Migrates and replicates data between heterogeneous or homogeneous sources.

Q: How do you run tests in a CI/CD pipeline on AWS?

A: Use CodeBuild or integrate third-party tools with CodePipeline or GitHub Actions.

Q: What is the advantage of using CodePipeline?

A: Automates build-test-deploy stages with native AWS service integrations.

Q: How do you rollback a failed Lambda deployment?

A: Use aliases and versions with automated deployment strategies like canary or linear.

Q: How can you isolate tenants in a SaaS model?

A: Use separate IAM roles, DynamoDB partition keys, or VPCs per tenant.

Q: What AWS service helps manage tenant-level access to shared data lake?

A: AWS Lake Formation with fine-grained permissions.

Q: How can you track changes in AWS infrastructure?

A: Use AWS Config and CloudTrail for compliance and audit trails.

Q: How to enforce tagging compliance across accounts?

A: Use AWS Config Rules and SCPs via AWS Organizations.

Q: Which service helps with compliance reports like PCI, SOC, HIPAA?

A: AWS Artifact provides downloadable compliance reports.

Q: What's the difference between CloudFormation and Terraform?

A: CloudFormation is AWS-native; Terraform is multi-cloud and uses HCL.

Q: How do you modularize Terraform code?

A: Use modules with separate files for `main.tf`, `variables.tf`, and `outputs.tf`.

Q: What tool helps prevent misconfigurations in Terraform?

A: `terraform validate`, `tfsec`, or Checkov.

GLOSSARY (CATEGORY-WISE)



Compute & Serverless

- **EC2 (Elastic Compute Cloud):** Scalable virtual servers in AWS. Offers flexibility in OS, instance type, storage, and networking.
- **Lambda:** Serverless compute service that runs code in response to events without provisioning or managing servers.
- **ECS (Elastic Container Service):** Managed container orchestration for Docker containers.
- **EKS (Elastic Kubernetes Service):** Managed Kubernetes service that simplifies running Kubernetes on AWS.
- **Fargate:** Serverless compute engine for containers that works with ECS and EKS.
- **Auto Scaling Group (ASG):** Automatically adjusts the number of EC2 instances based on demand.
- **Instance Types:** Categories of EC2 instances optimized for specific tasks (compute, memory, storage).
 - **t2.micro:** Burstable, low-cost instance type for light workloads.
 - **m5.large:** General-purpose instance with balanced CPU and memory.
- **launch_template:** Blueprint for launching EC2 instances with advanced configurations.
- **launch_configuration:** Older method for defining ASG parameters (now replaced by launch templates).



Storage & Data

- **S3 (Simple Storage Service):** Scalable object storage for data backup, archival, and web hosting.
- **EBS (Elastic Block Store):** Persistent block storage for EC2 instances.
- **EFS (Elastic File System):** Scalable file storage that can be mounted to multiple EC2 instances.
- **Glacier:** Long-term archival storage with low cost and retrieval latency options.

- **S3 IA (Infrequent Access):** Cheaper S3 storage for data accessed less frequently.
- **S3 Glacier Deep Archive:** Lowest-cost S3 storage tier for data rarely accessed.
- **aws s3 mb:** CLI command to make a new S3 bucket.
- **aws s3 cp:** CLI command to copy data to/from S3.
- **lifecycle rules:** Automatically transition or expire S3 objects based on age.
- **S3 versioning:** Keeps multiple versions of objects to protect against accidental deletions.
- **S3 CRR (Cross-Region Replication):** Replicates S3 objects to another region for disaster recovery.

Databases

- **RDS (Relational Database Service):** Managed SQL database with automated backups and patching.
- **Aurora:** High-performance, MySQL- and PostgreSQL-compatible database engine on RDS.
- **DynamoDB:** Fully managed NoSQL database known for low latency and scalability.
- **Redshift:** Managed data warehouse for analytics.
- **Timestream:** Time series database for IoT and operational applications.
- **QLDB (Quantum Ledger Database):** Immutable and cryptographically verifiable ledger database.
- **DocumentDB:** Managed document database service compatible with MongoDB.
- **SQL Engines:** Used in RDS and Aurora.
 - MySQL
 - PostgreSQL
 - MariaDB
- **DAX (DynamoDB Accelerator):** In-memory cache for DynamoDB to speed up read performance.
- **Streams:** Capture changes in DynamoDB tables for event-driven use cases.
- **TTL (Time to Live):** Auto-expire items in DynamoDB after a set period.
- **read replicas:** Read-only copies of a database for horizontal scaling.
- **multi-AZ:** High availability setup by replicating data across multiple Availability Zones.



Networking

- **VPC (Virtual Private Cloud):** Isolated network in AWS where you launch resources.
- **Subnet:** Divides VPC into smaller networks for organizing resources.
- **Internet Gateway:** Enables internet access for resources in a VPC.
- **NAT Gateway:** Provides outbound internet access to private subnet resources.
- **Route Table:** Defines how traffic is directed within a VPC.
- **Security Group (SG):** Virtual firewall controlling inbound and outbound traffic to resources.
- **Network ACL (NACL):** Stateless firewall for controlling traffic at subnet level.
- **Route 53:** Scalable Domain Name System (DNS) web service.
- **PrivateLink:** Securely connects VPCs to AWS services without going over the internet.
- **VPC Endpoints:** Private connection to AWS services within a VPC.
- **Transit Gateway:** Connects multiple VPCs and on-prem networks via a central hub.
- **VPC Peering:** Connects two VPCs to route traffic between them privately.



Security & Identity

- **IAM (Identity and Access Management):** Manage user access and permissions.
- **STS (Security Token Service):** Issues temporary, limited-privilege credentials.
- **Policies:** Define permissions in JSON to control access.
- **Roles:** Assign temporary access to AWS resources.
- **Groups:** Logical collection of IAM users with shared policies.
- **Users:** Individual accounts under IAM for accessing AWS.
- **ABAC (Attribute-Based Access Control):** Grant permissions based on user attributes (e.g., tags).
- **MFA (Multi-Factor Authentication):** Adds an extra layer of security during login.
- **AWS Secrets Manager:** Securely stores and retrieves secrets like API keys.
- **SSM Parameter Store:** Stores configuration and secrets as parameter values.
- **KMS (Key Management Service):** Create and control encryption keys.
- **SSE-KMS:** Server-side encryption using KMS keys.

- **SSE-S3**: Server-side encryption with S3-managed keys.
- **CMK (Customer Master Key)**: Customer-defined key in KMS.
- **AWS Shield**: DDoS protection for AWS services.
- **AWS WAF (Web Application Firewall)**: Protects web applications from common threats.
- **Firewall Manager**: Centralizes firewall policy management across accounts.
- **GuardDuty**: Threat detection using ML and threat intelligence.
- **Macie**: Detects and protects sensitive data using ML.
- **Inspector**: Security assessment service for EC2 and containers.
- **Audit Manager**: Automates evidence collection for audits.
- **Security Hub**: Aggregates security findings from various AWS services.

Infrastructure as Code (IaC)

- **Terraform**: Open-source IaC tool to define AWS resources in HCL.
- **CloudFormation**: AWS-native IaC tool using YAML/JSON templates.
- **CodePipeline**: CI/CD service to automate build, test, and deployment workflows.
- **Resource blocks (e.g., `resource "aws_instance"`)**: Define infrastructure resources in Terraform.



Command Line Interface (CLI) Commands

- **`aws ec2 run-instances`**: Launch new EC2 instances.
- **`aws s3 cp`**: Copy data to/from S3.
- **`aws lambda create-function`**: Deploy a new Lambda function.
- **`aws dynamodb create-table`**: Create new DynamoDB table.
- **`aws secretsmanager create-secret`**: Store a new secret.
- **`aws cloudwatch put-metric-data`**: Publish custom metrics to CloudWatch.
- **`aws kms create-key`**: Create a new KMS key.
- **`aws route53 create-hosted-zone`**: Set up a new DNS zone.
- **`aws budgets create-budget`**: Define budget limits and alerts.

- **aws logs create-log-group/create-log-stream**: Set up CloudWatch logging structures.



SDK Usage (Python/Boto3)

- **boto3.client('s3')**: Connect to S3 using Boto3.
- **boto3.client('secretsmanager')**: Connect to Secrets Manager.
- **client.get_secret_value()**: Retrieve secret from Secrets Manager.
- **s3.create_bucket()**: Create an S3 bucket programmatically.



Monitoring & Observability

- **CloudWatch**: Central monitoring service for metrics, logs, and alarms.
- **X-Ray**: Traces requests through applications to identify bottlenecks.
- **CloudTrail**: Logs all API calls for auditing.
- **VPC Flow Logs**: Capture network traffic metadata in VPC.
- **Reachability Analyzer**: Checks connectivity between AWS resources.
- **put-dashboard**: Create a CloudWatch dashboard.
- **put-metric-data**: Push custom metrics.
- **create-log-group/stream**: Set up log storage in CloudWatch.
- **aws cloudwatch create-alarm**: Define alarms for metrics.
- **EventBridge**: Event bus for building event-driven architectures.



Scalability & Resilience

- **Auto Scaling**: Adjusts resource capacity based on demand.
- **Horizontal Scaling**: Add more instances to handle increased load.
- **Vertical Scaling**: Upgrade instance types for more CPU/memory.
- **Exponential Backoff**: Retry mechanism with increasing delay.
- **Retry Logic**: Code that retries failed operations.
- **Idempotency Token**: Prevents duplication in repeated requests.
- **Dead Letter Queue (DLQ)**: Stores failed messages for review.

- **Pilot Light:** DR strategy with minimal active infrastructure.
- **Warm Standby:** DR setup with scaled-down replica ready to scale up.



Containers & Microservices

- **App Mesh:** Service mesh for controlling service-to-service communication.
- **Service Discovery:** Automatically detect and connect to services.
- **Docker:** Container platform to package applications.
- **Kubernetes:** Open-source container orchestration platform.
- **pod autoscaler/HPA:** Scales pods based on metrics (e.g., CPU).
- **VPA (Vertical Pod Autoscaler):** Adjusts CPU/memory limits of pods.



Compliance & Governance

- **SCP (Service Control Policy):** Controls what services/org units can do in AWS Org.
- **AWS Organizations:** Centrally manage multiple AWS accounts.
- **IAM Analyzer:** Identify resources shared with external entities.
- **Trusted Advisor:** Offers best practice checks for cost, performance, and security.
- **Well-Architected Tool:** Evaluates architectures against AWS best practices.



Cost Optimization

- **AWS Budgets:** Set cost or usage budgets and receive alerts.
- **Cost Explorer:** Visualize cost and usage trends.
- **Compute Optimizer:** Recommends right-sizing for cost savings.
- **S3 Storage Classes:** Different pricing tiers based on usage patterns.
- **Graviton:** ARM-based processor for better price-performance.
- **Fargate Spot:** Discounted Fargate compute for fault-tolerant workloads.
- **EBS rightsizing:** Optimize EBS volume types and sizes.
- **Reserved Instances (RI):** Commit to usage for 1 or 3 years to save cost.
- **Spot Instances:** Buy spare EC2 capacity at discount.



Testing Tools

- **AWS Fault Injection Simulator (FIS):** Chaos engineering to test system resilience.
- **AWS Resilience Hub:** Central hub to manage application resilience.
- **AWS Chaos Engineering:** Practice of testing systems by injecting faults.